

MASSMORE TECHNICAL GUIDE

คู่มือการใช้งาน

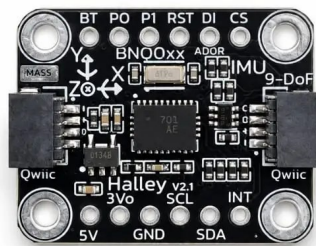
Massmore Halley V2.1

BNO085 / BNO086 9-DOF IMU Sensor Module (SKU-1010)

9-DOF IMU SENSOR

BNO055 • BNO085 • BNO086

HALLEY V2.1 ชิปแก้จาก **CEVA & BOSCH**



ไลบรารี Massmore_BNO08x เวอร์ชัน 2.1.1

Arduino IDE • PlatformIO • ESP32 / ESP32-S3 (Arduino-ESP32 Core 3.x) • Arduino Nano
I2C • SPI • UART-SHTP • UART-RVC

จัดทำโดย Massmore Biz Co., Ltd.

www.massmore.shop

ฉบับปรับปรุง 6 ตุลาคม 2569

คำนำ

คู่มือฉบับนี้จัดทำขึ้นเพื่อเป็นแนวทางการใช้งานโมดูลเซนเซอร์ Massmore Halley V2.1 ชิป BNO085 / BNO086 ร่วมกับไลบรารี Massmore BNO08x เวอร์ชัน 2.1.1 ตั้งแต่การรู้จักบอร์ด การต่อวงจร การติดตั้งซอฟต์แวร์ ไปจนถึงการใช้งานขั้นสูงผ่าน SPI และ UART การ calibrate การตรวจสอบความเป็นของแท้ของชิป และการแก้ไขปัญหา

เนื้อหาแต่ละบทเรียงตามลำดับการเรียนรู้ ประกอบด้วยสิ่งที่ต้องเตรียม การต่อวงจร ขั้นตอนการตั้งค่า ตัวอย่างโค้ด ผลลัพธ์ที่ควรเห็น และวิธีแก้ไขเมื่อไม่ได้ผล ผลลัพธ์ที่ระบุว่าเป็น “ผลจริง” คัดลอกจาก Serial Monitor ระหว่างการทดสอบกับบอร์ดจริง (ชิป BNO086 เฟิร์มแวร์ 3.12.6) ส่วนที่ระบุว่าเป็น “รูปแบบผลลัพธ์” แสดงรูปแบบของข้อความ ตัวเลขจริงขึ้นกับท่าวางบอร์ดและสภาพแวดล้อม

ข้อมูลทางเทคนิคของชิปอ้างอิงจาก datasheet และเอกสารของผู้ผลิตชิปเป็นหลัก ผู้อ่านที่เพิ่งเริ่มต้นควรอ่านบทที่ 1 ถึงบทที่ 4 ตามลำดับ แล้วจึงเลือกอ่านบทที่ตรงกับงานของตน ส่วนผู้ที่ต้องการนำไปใช้ในผลิตภัณฑ์ควรอ่านบทที่ 7, 11 และ 13 เพิ่มเติม

Massmore Biz Co., Ltd.

ตุลาคม 2569

สารบัญ

	หน้า
บทที่ 1 รู้จักบอร์ด Massmore Halley V2.1	1
1.1 ภาพรวมของผลิตภัณฑ์	1
1.2 คุณสมบัติเด่น	2
1.3 ข้อมูลจำเพาะของบอร์ด	3
1.4 ข้อมูลจำเพาะของชิป BNO08x	4
1.5 ความแตกต่างระหว่าง BNO085 และ BNO086	5
1.6 เปรียบเทียบกับ BNO055	6
1.7 หลักการ sensor fusion และการแทนทิศทาง	7
บทที่ 2 ขาต่อใช้งานและการเลือกโหมด	8
2.1 ตำแหน่งและหน้าที่ของขา	8
2.2 หน้าที่ของ pad ในแต่ละโหมด	9
2.3 การเลือกโหมดด้วย P0 / P1	9
2.4 I2C address ไฟเลี้ยง และระดับแรงดัน	10
2.5 รูปแบบการต่อใช้งาน	10
บทที่ 3 การติดตั้งซอฟต์แวร์	12
3.1 ติดตั้ง Arduino IDE และ ESP32 board package	12
3.2 ติดตั้งไลบรารี Massmore_BNO08x	12
3.3 การตั้งค่าบอร์ดใน Arduino IDE	12
3.4 ใช้งานด้วย PlatformIO (VS Code)	13
บทที่ 4 เริ่มต้นใช้งานผ่าน I2C	14
4.1 อุปกรณ์ที่ต้องเตรียม	14
4.2 การต่อวงจร	14
4.3 อัปโหลดตัวอย่าง 01_BasicRead	15
4.4 ย้ายขาและใช้ I2C bus ที่สอง (ตัวอย่าง 02)	17
บทที่ 5 วัดมุมโดยไม่ต้อง calibrate	18
5.1 ความต่างจาก Rotation Vector	18
5.2 การต่อวงจรและการใช้งาน	18
บทที่ 6 Non-blocking API สำหรับงานหลายภารกิจ	20
6.1 ลำดับการทำงานของ FSM API	20
6.2 อัตรา report	20
6.3 ผลลัพธ์และการอ่านค่าสถิติ	21
บทที่ 7 Calibration และ Tare	22
7.1 Calibration กับ Tare ต่างกันอย่างไร	22
7.2 ขั้นตอน calibration	22
7.3 คำสั่งทั้งหมด	23
7.4 สภาพแวดล้อมที่เหมาะสม	23

บทที่ 8 Euler angles และเข็มทิศ.....	24
8.1 แกนอ้างอิงและการคำนวณ heading.....	24
8.2 ผลลัพธ์.....	24
บทที่ 9 SPI สำหรับงานอัตราสูง.....	25
9.1 การต่อวงจร.....	25
9.2 โค้ดและผลลัพธ์.....	26
บทที่ 10 การเชื่อมต่อแบบ UART.....	27
10.1 UART-RVC (ตัวอย่าง 08).....	27
10.2 UART-SHTP (ตัวอย่าง 09).....	28
บทที่ 11 การตรวจสอบของแท่งและ Factory Test.....	29
11.1 ฟังก์ชันตรวจสอบตัวตนชิป.....	29
11.2 การใช้งาน Factory Test.....	29
11.3 เฟิร์มแวร์สำเร็จรูป.....	31
บทที่ 12 สรุป API ของไลบรารี.....	32
12.1 การเริ่มต้น.....	32
12.2 Simple Blocking API.....	32
12.3 Non-blocking FSM API.....	32
12.4 Calibration, Tare, พลังงาน และ FRS.....	33
12.5 รหัสข้อผิดพลาด.....	33
บทที่ 13 การแก้ไขปัญหา.....	35
13.1 ปัญหาการเชื่อมต่อ.....	35
13.2 ปัญหาค่าที่อ่านได้.....	35
13.3 ปัญหาจากเครื่องมือพัฒนา.....	36
13.4 รายการตรวจสอบก่อนนำไปใช้ในผลิตภัณฑ์.....	36
ภาคผนวก ก ขาที่ตัวอย่างแต่ละชุดใช้.....	37
ภาคผนวก ข ผลการทดสอบและประวัติการเปลี่ยนแปลง.....	38
ข.1 MCU ที่รองรับ.....	38
ข.2 ผลทดสอบกับบอร์ดจริง.....	38
ข.3 ประวัติการเปลี่ยนแปลง.....	39
ภาคผนวก ค เอกสารอ้างอิงและแหล่งข้อมูล.....	40
ค.1 ไลบรารีและสินค้า.....	40
ค.2 เอกสารจากผู้ผลิตชิป.....	40
ค.3 เครื่องมือพัฒนา.....	40
ค.4 บทความวิชาการพื้นฐาน.....	40

บทที่ 1

รู้จักบอร์ด Massmore Halley V2.1

1.1 ภาพรวมของผลิตภัณฑ์

Massmore Halley V2.1 (SKU-1010) เป็นโมดูลเซนเซอร์วัดการเคลื่อนที่ 9 แกน (9-DOF IMU) ที่ใช้ชิป **BNO085** หรือ **BNO086** ภายในชิปรวม accelerometer, gyroscope และ magnetometer อย่างละ 3 แกน เข้ากับไมโครคอนโทรลเลอร์ 32 บิต (ARM Cortex-M0+) ที่รันเฟิร์มแวร์ SH-2 MotionEngine ไว้ในแพ็คเกจเดียว

จุดต่างสำคัญจาก IMU ทั่วไปคือ BNO08x ประมวลผล **sensor fusion** ภายในชิปเอง แล้วส่งผลลัพธ์ที่พร้อมใช้ เช่น quaternion, rotation vector, gravity และ linear acceleration ให้ไมโครคอนโทรลเลอร์หลัก (host) โดยตรง ผู้ใช้จึงไม่ต้องเขียนหรือจูนตัวกรอง Kalman หรือ Madgwick เอง และลดภาระการคำนวณของ ESP32 ได้มาก

ไลบรารี **Massmore BNO08x เวอร์ชัน 2.1.1** สื่อสารด้วยโปรโตคอล SH-2/SHTP ให้ทั้งหมด รองรับการเชื่อมต่อ I2C, SPI, UART-SHTP และ UART-RVC มีตัวอย่าง 10 ชุดสำหรับ Arduino IDE และโปรเจกต์ PlatformIO ที่เปิดแล้ว build ได้ทันที

HALLEY V2.1

โมดูล AR VR 9-DOF IMU Sensor



 PCB สัดๆ บอร์ด Halley V2.1 PCB สัดๆ ขนาดมาตรฐาน	 9 แกน IMU Accelerometer + Gyroscope + Magnetometer พร้อม ARM Cortex-M0+	 Sensor Fusion ส่งค่า Quaternion, Euler และ Linear Acceleration ได้โดยตรง
 ภาระ MCU ช่วยลดภาระประมวลผลของไมโครคอนโทรลเลอร์	 Auto Calibration สอนเทียบอัตโนมัติ	 Qwiic / STEMMA QT Qwiic / STEMMA QT 2 ช่องต่อพ่วงได้ไม่จำกัด
 3.3V / 5V ไฟเลี้ยง 3.3V/5V • SDA/SCL มี Level Shifter	 PS0 / PS1 / ADDR เลือกโหมดด้วย jumper PS0/PS1 และแอดเดรสด้วย ADDR	 Massmore Library 8 ปีกี่ ตรวจสอบ Product ID ผ่านไลบรารี Massmore

รูปที่ 1.1 จุดเด่นของ Massmore Halley V2.1

1.2 คุณสมบัติเด่น

ตารางที่ 1.1 ข้อมูลทิศทาง (orientation) ที่ชิปคำนวณให้

Output	เซนเซอร์ที่ใช้	อัตราสูงสุด	เหมาะกับงาน
Rotation Vector	Accel + Gyro + Mag	400 Hz	เข็มทิศ หุ่นยนต์ที่ต้องรู้ทิศทางเหนือ
Game Rotation Vector	Accel + Gyro	400 Hz	วัดมุม gimbal งานไกล้มอเตอร์ไม่ต้อง calibrate
Geomagnetic Rotation Vector	Accel + Mag	90 Hz	งานประหยัดพลังงานที่เคลื่อนที่ช้า
AR/VR Stabilized RV / Game RV	9 แกน / 6 แกน	400 Hz	ภาพสามมิติที่ต้องการความนิ่ง
Gyro-Integrated Rotation Vector	Gyro (integrate)	1000 Hz	head tracking, control loop อัตราสูง

ตารางที่ 1.2 ข้อมูลการเคลื่อนที่และหน่วยในไลบรารี

Output	หน่วย	อัตราสูงสุด
Accelerometer	m/s ² (รวม gravity)	500 Hz
Gyroscope	rad/s (<code>getGyroDeg()</code> ให้ค่าเป็น °/s)	400 Hz
Magnetometer	μT	100 Hz
Linear Acceleration	m/s ² (ตัด gravity แล้ว)	400 Hz
Gravity	m/s ²	400 Hz
Raw Accelerometer / Gyroscope / Magnetometer	ค่า ADC (counts)	ตาม report

อัตราในตารางเป็นค่าสูงสุดราย report ตาม datasheet ไม่ได้หมายความว่าเปิดทุก report ที่อัตราสูงสุดพร้อมกันได้

การตรวจจับกิจกรรมและท่าทาง

- Step Counter และ Step Detector
- Personal Activity Classifier, Stability Classifier และ Stability Detector
- Tap / Double Tap, Shake, Flip, Pickup, Tilt, Pocket, Circle Detector
- Significant Motion Detector และ Sleep Detector

ความสามารถของไลบรารี

- **API สองระดับ:** Simple Blocking API (`readAll()`) สำหรับผู้เริ่มต้นและ Arduino Nano และ Non-blocking FSM API (`update()` / `isDataReady()` / `getReadings()`) สำหรับงานหลายภารกิจหรือ RTOS
- **ตรวจสอบตัวตนชิป:** `verifyChipID()`, `getChipModel()`, `getSerialNumber()` และ `isGenuine()`

- Calibration, Tare, Save DCD, อ่าน/เขียน FRS (Flash Record System), sleep / wake, soft / hardware reset
- ไม่มี dependency ภายนอก ไม่ใช่ dynamic allocation และไม่กำหนดขา GPIO ไว้ในไลบรารี ผู้ใช้กำหนดขาใน sketch เองทั้งหมด
- ตัวอย่าง Factory Test พร้อมเฟิร์มแวร์สำเร็จรูปสำหรับตรวจสอบรับสินค้า

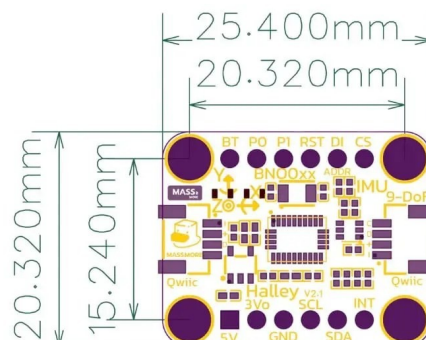
1.3 ข้อมูลจำเพาะของบอร์ด

ตารางที่ 1.3 ข้อมูลจำเพาะของบอร์ด Massmore Halley V2.1 (SKU-1010)

รายการ	ข้อมูลจำเพาะ
ชิปเซนเซอร์	BNO085 หรือ BNO086 (SH-2 MotionEngine)
การเชื่อมต่อ	I2C (ค่าเริ่มต้น), SPI Mode 3 ≤ 3 MHz, UART-SHTP 3 Mbaud, UART-RVC 115200 baud
I2C address	0x4A (ค่าเริ่มต้น, DI ปลอยลอย) / 0x4B (DI ต่อ 3Vo)
ไฟเลี้ยง	3.3 V ที่ pad 3Vo หรือ 5 V ที่ pad 5V (มีวงจรควบคุมแรงดัน 3.3 V บนบอร์ด)
ระดับแรงดันลอจิก	3.3 V — เฉพาะ SDA / SCL มีวงจรแปลงระดับแรงดันสองทิศทาง ต่อกับ host 5 V ได้ ขาอื่นเป็น 3.3 V เท่านั้น
ตัวต้านทาน pull-up	SDA / SCL มี pull-up บนบอร์ด (datasheet แนะนำ 2–4 k Ω)
ขั้วต่อ	Qwiic 4 ขา จำนวน 2 ช่อง (ฝั่ง 3.3 V) และ pad สำหรับบัดกรีขา
การเลือกโหมด	ต่อสายที่ pad P0 / P1 หรือบัดกรี jumper PS0 / PS1 ด้านหลังบอร์ด
ขนาดบอร์ด	25.40 × 20.32 mm ระยะรูยึด 20.32 × 15.24 mm

HALLEY V2.1

โมดูล AR VR 9-DOF IMU Sensor



รูปที่ 1.2 ขนาดบอร์ดและระยะรูยึด

1.4 ข้อมูลจำเพาะของชิป BNO08x

ตารางที่ 1.4 ข้อมูลทั่วไปของชิป

รายการ	ค่า
ประเภท	9-DOF sensor fusion System-in-Package
หน่วยประมวลผลภายใน	ARM Cortex-M0+ 32 บิต
แพ็คเกจ	LGA 28 ขา 5.2 × 3.8 × 1.1 mm
VDD / VDDIO ของชิป	2.4–3.6 V / 1.7–3.6 V
อุณหภูมิใช้งาน	–40 ถึง +85 °C
I2C	100 / 400 kHz, address 0x4A หรือ 0x4B, มี clock stretching
SPI	4 สาย, Mode 3 (CPOL = 1, CPHA = 1), สูงสุด 3 MHz
UART-SHTP	3,000,000 baud 8N1
UART-RVC	115200 baud 8N1, ส่งข้อมูลเอง 100 Hz

ตารางที่ 1.5 ความแม่นยำของ sensor fusion ตาม datasheet

Output	เงื่อนไข	ค่า
Rotation Vector	Dynamic / Static rotation error	3.5° / 2.0°
Game Rotation Vector	Dynamic / Static non-heading error	2.5° / 1.5°
Game Rotation Vector	Heading drift	0.5°/นาที
Geomagnetic Rotation Vector	Dynamic rotation error	4.5°
Gravity	Static angle error	1.5°
Linear Acceleration / Accelerometer	Dynamic accuracy	0.35 / 0.3 m/s ²
Gyroscope / Magnetometer	Dynamic accuracy	3.1 °/s / 1.4 µT

หมายเหตุ: ผู้ผลิตชิประบุว่าในการใช้งานจริง Rotation Vector มักคลาดเคลื่อนราว 5° และ Geomagnetic Rotation Vector ราว 10° ขึ้นกับสภาพสนามแม่เหล็กรอบตัว จึงควรออกแบบค่าเผื่อจากค่าใช้งานจริง ไม่ยึดเฉพาะตัวเลขในตาราง

ตารางที่ 1.6 เวลาหน่วง (latency) ของข้อมูลทิศทาง

Output	ที่ 100 Hz	ที่ 200 Hz
Rotation Vector / Game RV / Gyro-Integrated RV	6.6 ms	3.7 ms

ตารางที่ 1.7 ตัวอย่างการใช้พลังงานของ BNO086 (SPI, VDDIO 3.0 V, VDD 3.3 V)

การตั้งค่า	กำลังไฟฟ้า
Idle	0.17 mW
9-axis fusion 100 Hz	34.29 mW
9-axis fusion 400 Hz	44.40 mW
Gyro-Integrated RV 1000 Hz	45.27 mW
Accelerometer 125 Hz	3.44 mW
Step counter 31.25 Hz	1.54 mW

1.5 ความแตกต่างระหว่าง BNO085 และ BNO086

BNO086 ใช้แทน BNO085 ได้โดยตรง (drop-in replacement) ขา การเชื่อมต่อ และ report ที่ใช้ร่วมกันเหมือนกันทุกประการ จึงใช้การต่อสายและ API ของไลบรารีชุดเดียวกันได้ทันที ส่วนต่างที่สำคัญมีดังนี้

ตารางที่ 1.8 เปรียบเทียบ BNO085 และ BNO086

หัวข้อ	BNO085	BNO086	ผลต่อการใช้งาน
Pinout และ report พื้นฐาน	รองรับ	รองรับ	ใช้โค้ดเดียวกัน
Accelerometer ใน fusion	มาตรฐาน	14 บิต	ความละเอียดสูงขึ้นโดยไม่ต้องแก้ไขโค้ด
กำลังไฟฟ้าขณะ idle	≈ 0.39 mW	≈ 0.17 mW	BNO086 เหมาะกับงานที่ idle นาน
Report เฉพาะรุ่น (0x2B–0x2D)	ไม่มี	มี	ใช้แยกรุ่นชิปใน <code>getChipModel()</code>
UART-RVC motion intent / request	ส่งวนไว้	รองรับ	อ่านจาก <code>motionIntent</code> , <code>motionRequest</code>

วิธีดูว่าบอร์ดที่ได้รับเป็นรุ่นใด: เรียก

`Massmore_BNO08x::chipModelToString(imu.getChipModel())` ซึ่งคืนค่า "BNO085", "BNO086" หรือ "BNO08x" (ระบุไม่ได้) หรือดูบรรทัด `#CHIP` ในผลของตัวอย่าง `10_Factory_Test` ด้านหลังบอร์ดมีช่องทำเครื่องหมายรุ่นชิปไว้ด้วย

İlqə AR VR 9-DOF IMU Sensor



รูปที่ 1.3 ด้านหน้าและด้านหลังของบอร์ด (ด้านหลังมีตารางโหมดและ jumper PS0 / PS1)

1.6 เปรียบเทียบกับ BNO055

บอร์ดตระกูล Halley V2.1 รองรับทั้งชิป BNO055 และ BNO08x บน footprint เดียวกัน แต่ทั้งสองรุ่นใช้โปรโตคอลต่างกันโดยสิ้นเชิง ไลบรารีในคู่มือนี้ใช้กับ BNO085 / BNO086 เท่านั้น จุดต่างหลักคือ BNO08x รองรับ SPI และ UART-SHTP เพิ่ม และให้ข้อมูลทิศทางได้ถึง 400 Hz (Gyro-Integrated RV ถึง 1 kHz) ขณะที่ BNO055 ให้ผล fusion สูงสุด 100 Hz

HALLEY V2.1

โมดูล AR VR 9-DOF IMU Sensor



BNO055
(Bosch Sensortec)

BNO0xx 9-DoF

Adv. Inertial
9DoF Accel/Gyro/Mag
I2C/UART/SPI Interface
www.massmore.shop

Interface (การเชื่อมต่อ)

I ² C	I ² C	สูงสุด 400 kHz
UART	UART	115200 bps
ψ	HID-over-I ² C	

อัตราการอัปเดตข้อมูล (Data Update Rate)

NDOF / IMU Fusion
สูงสุด 100 Hz

BNO085 / BNO086 (BNO08x)
(CEVA)

BNO0xx 9-DoF

Adv. Inertial
9DoF Accel/Gyro/Mag
I2C/UART/SPI Interface
www.massmore.shop

Interface (การเชื่อมต่อ)

I ² C	I ² C	สูงสุด 400 kHz Massmore library สูงสุด 100 kHz เพื่อควบคุมการยืดตัวของนาฬิกา
SPI	SPI (Mode 3)	สูงสุด 3 MHz
UART-SHTP	UART-SHTP	3 Mbaud
UART-RVC	UART-RVC	115200 bps

อัตราการอัปเดตข้อมูล (Data / Report Rate)

Rotation Vector (RV)	สูงสุด 400 Hz
Gyro-Integrated Rotation Vector	สูงสุด 1 kHz
UART-RVC	100 Hz

สรุปอัตราการอัปเดตข้อมูล (โดยย่อ)

BNO055 Fusion	BNO08x Rotation Vector (RV)	BNO08x Gyro-Integrated RV
100 Hz	400 Hz	1 kHz

**BNO08x รองรับ Interface มากกว่า
และอัปเดตข้อมูล Orientation ได้เร็วกว่า BNO055**
เหมาะสำหรับ Robot / AR-VR / Motion Tracking / High-rate Control

9-DoF IMU Sensor
Compact & Easy to Use
Support I2C / UART / SPI
For Makers & Professionals
อ้างอิง: Bosch BNO055 Datasheet + Massmore BNO08x v2.1.0 GitHub

รูปที่ 1.4 เปรียบเทียบการเชื่อมต่อและอัตราข้อมูลของ BNO055 กับ BNO085 / BNO086

1.7 หลักการ sensor fusion และการแทนทิศทาง

ตารางที่ 1.9 จุดเด่นและข้อจำกัดของเซนเซอร์แต่ละชนิด

เซนเซอร์	จุดเด่น	ข้อจำกัด
Accelerometer	รู้ทิศทางของแรงโน้มถ่วง วัดความเร่ง	ไวต่อการสั่นและการเคลื่อนที่ บอกทิศเหนือไม่ได้
Gyroscope	ตอบสนองเร็ว รวดเร็ว	เมื่อ integrate นาน ๆ เกิด drift สะสม
Magnetometer	อ้างอิงทิศเหนือแม่เหล็ก	ถูกรบกวนจากมอเตอร์ เหล็ก แม่เหล็ก และสายกระแสสูง

MotionEngine รวมข้อมูลทั้งสามชนิดเพื่อให้ได้ทิศทางที่เสถียรมากกว่าการใช้เซนเซอร์ตัวเดียว ถ้าบริเวณใช้งานมีสนามแม่เหล็กรบกวน ให้เปลี่ยนจาก Rotation Vector เป็น Game Rotation Vector เพื่อป้องกัน yaw กระโดด โดยยอมให้ yaw drift อย่างช้า ๆ แทน

ชิปส่งทิศทางเป็น **quaternion** (i, j, k, real) ซึ่งไม่เกิดปัญหา gimbal lock ไลบรารีแปลงเป็น **Euler angles** (roll, pitch, yaw หน่วยองศา) ให้ง่ายต่อการเก็บและคำนวณภายในด้วย quaternion แล้วแปลงเป็น Euler เฉพาะตอนแสดงผล

```
Massmore_BNO08x_quat_t q = imu.getQuaternion(); // i, j, k, real
Massmore_BNO08x_euler_t e = imu.getEulerDeg(); // roll, pitch, yaw (±180)
float h = imu.getHeadingDeg(); // yaw ในช่วง 0..360
```

บทที่ 2

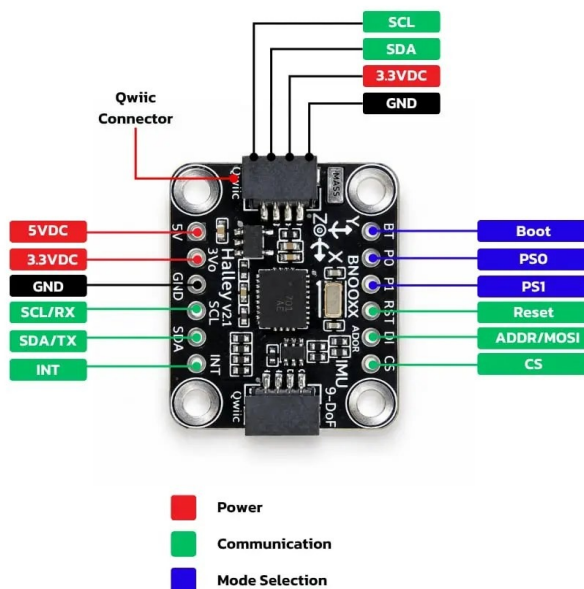
การใช้งานและการเลือกโหมด

2.1 ตำแหน่งและหน้าที่ของขา

ชื่อขาในคู่มือ ตัวอย่างโค้ด และ README ใช้ชื่อที่พิมพ์บนบอร์ด (silkscreen) เป็นหลัก แฉวล่างของบอร์ดคือ 5V 3Vo GND SCL SDA INT และแฉวบนคือ BT P0 P1 และแฉวบนคือ BT P0 P1 RST DI CS

HALLEY V2.1

โมดูล AR VR 9-DOF IMU Sensor



รูปที่ 2.1 Pinout ของ Massmore Halley V2.1 (แดง = ไฟเลี้ยง, เขียว = การสื่อสาร, น้ำเงิน = เลือกโหมด)

ตารางที่ 2.1 หน้าที่ของแต่ละขา

Pad	ชื่อในเอกสารชิป	หน้าที่	หมายเหตุ
5V	—	ไฟเข้า 5 V	ผ่านวงจรควบคุมแรงดัน 3.3 V บนบอร์ด
3Vo	VDD / VDDIO	ไฟ 3.3 V เข้า / ออก	จ่าย 3.3 V เข้าที่นี่ได้ หรือดึง 3.3 V ออกไปใช้ (กระแสต่ำ)
GND	GND	กราวด์	ต้องต่อร่วมกับ host
SDA	SDA · TX · MISO	I2C data	ผ่านวงจรแปลงระดับแรงดัน มี pull-up
SCL	SCL · RX · SCK	I2C clock	ผ่านวงจรแปลงระดับแรงดัน มี pull-up
INT	H_INTN	แจ้งว่ามีข้อมูล (active-low)	3.3 V — แนะนำให้ต่อ

Pad	ชื่อในเอกสารชิป	หน้าที่	หมายเหตุ
RST	NRST	รีเซ็ตชิป (active-low)	3.3 V — แนะนำให้ต่อทุกโหมด
DI	SA0 · MOSI	เลือก address / SPI MOSI	ปล่อยลอย = 0x4A, ต่อ 3Vo = 0x4B
CS	H_CSN	SPI chip select	3.3 V เท่านั้น
BT	BOOTN	เลือก bootloader	ห้ามดึง LOW ตอน reset
P0	PS0	เลือกโหมด บิต 0 / WAKE ในโหมด SPI	3.3 V เท่านั้น
P1	PS1	เลือกโหมด บิต 1	3.3 V เท่านั้น

ข้อควรระวัง: ขาทุกขาออกจาก SDA และ SCL เป็นลอจิก 3.3 V ล้วน ห้ามป้อนสัญญาณ 5 V เข้าโดยตรง ถ้าใช้กับ Arduino Nano (5 V) ต้องผ่านวงจรแปลงระดับแรงดัน
ขั้วต่อ Qwiic ทั้งสองช่องเป็นฝั่ง 3.3 V ต่อตรงกับชิป ไม่ผ่านวงจรแปลงระดับแรงดัน

2.2 หน้าที่ของ pad ในแต่ละโหมด

ตารางที่ 2.2 หน้าที่ของ pad เมื่อใช้งานแต่ละโหมด

Pad	I2C	SPI	UART-SHTP	UART-RVC
SDA	SDA	MISO (ชิป → MCU)	TX ของชิป → RX ของ MCU	TX ของชิป → RX ของ MCU
SCL	SCL	SCK	RX ของชิป ← TX ของ MCU	ไม่ใช้
DI	address (ลอย = 0x4A)	MOSI (MCU → ชิป)	ไม่ใช้	ไม่ใช้
CS	ไม่ใช้	CS	ไม่ใช้	ไม่ใช้
INT	ไม่บังคับ	จำเป็น	ไม่บังคับ	ไม่ใช้
RST	แนะนำ	จำเป็น	แนะนำ	แนะนำ
P0	LOW	WAKE → GPIO	LOW	HIGH
P1	LOW	HIGH	HIGH	LOW
BT	ปล่อยลอย	ปล่อยลอย	ปล่อยลอย	ปล่อยลอย

2.3 การเลือกโหมดด้วย P0 / P1

ชิปอ่านระดับสัญญาณของ P1 (PS1) และ P0 (PS0) เฉพาะตอนพ้นจากการรีเซ็ตเท่านั้น เลือกโหมดได้สองวิธี คือต่อสายที่ pad P0 / P1 หรือบัดกรีปิด jumper PS0 / PS1 ด้านหลังบอร์ด (บัดกรีปิด = 1)

ตารางที่ 2.3 ตารางเลือกโหมด

โหมด	P1 (PS1)	P0 (PS0)	ชื่อบนบอร์ด	ความเร็ว	ตัวอย่าง
I2C (ค่าเริ่มต้น)	0	0	I2C	100 kHz (แนะนำ)	01–06, 10
SPI	1	1	SPI	≤ 3 MHz, Mode 3	07
UART-SHTP	1	0	UART	3,000,000 baud	09
UART-RVC	0	1	UART+	115200 baud, 100 Hz	08

สำคัญ: ด้านหลังบอร์ดพิมพ์ชื่อโหมดแบบย่อ: **UART** หมายถึง **UART-SHTP** และ **UART+** หมายถึง **UART-RVC**

โหมด SPI ห้ามบัดกรีปิด jumper PS0 เพราะ P0 ต้องต่อเข้า GPIO ของ MCU เพื่อใช้เป็นขา WAKE (รายละเอียดในบทที่ 9)

ขั้นตอนเปลี่ยนโหมดที่ถูกต้อง

1. ถอดไฟเซนเซอร์ หรือเตรียมขา **RST** ต่อเข้ากับ MCU
2. ต่อ **P0** / **P1** หรือบัดกรี jumper ตามตารางที่ 2.3
3. รีเซ็ตชิป — ถ้าต่อ **RST** ไว้ ตัวอย่างทุกชุดจะรีเซ็ตชิปให้เองตอน `begin()` ถ้าไม่ได้ ต่อให้ถอดไฟเซนเซอร์แล้วเสียบใหม่
4. อัปโหลดตัวอย่างของโหมดนั้น

ข้อควรระวัง: การกดปุ่มรีเซ็ตที่บอร์ด MCU ไม่ได้รีเซ็ตเซนเซอร์ ชิปปะยังอยู่ในโหมดเดิม จนกว่าจะรีเซ็ตที่ขา **RST** หรือตัดไฟ

2.4 I2C address ไฟเลี้ยง และระดับแรงดัน

- **I2C address:** DI ปลั๊กลอย = **0x4A** (ค่าเริ่มต้นของบอร์ด) ต่อ DI เข้า **3Vo** = **0x4B** ถ้า `begin()` หา address ที่ระบุไม่พบ ไบรารีจะลองอีก address ให้อัตโนมัติ
- **ไฟเลี้ยง:** จ่าย 3.3 V ที่ **3Vo** หรือ 5 V ที่ **5V** อย่างใดอย่างหนึ่ง
- **Pull-up:** SDA / SCL มี pull-up บนบอร์ดแล้ว ไม่ต้องต่อเพิ่ม
- **ความเร็ว I2C:** ชิปปะใช้ clock stretching ให้เริ่มที่ 100 kHz ซึ่งเสถียรที่สุด
- **ขา BT:** ถ้าถูกดึง LOW ตอนรีเซ็ต ชิปปะจะเข้า bootloader และตอบที่ address **0x28** / **0x29** แทน

2.5 รูปแบบการต่อใช้งาน

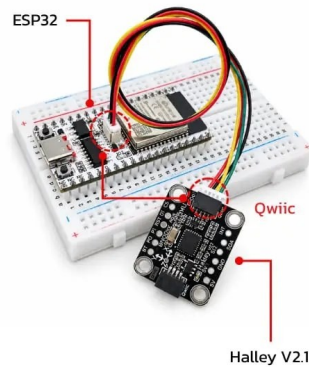
การต่อใช้งานมีสองรูปแบบหลัก แบบที่ 1 เสียบสาย Qwiic เข้ากับบอร์ด MCU ที่มี ขั้วต่อ Qwiic ได้ทันทีโดยไม่ต้องบัดกรี เหมาะกับโหมด I2C และงานต้นแบบ แบบที่ 2 บัดกรีขา pin header แล้วเสียบลง breadboard ใช้ได้ทุกโหมด และจำเป็นสำหรับ SPI, UART หรือเมื่อต้องต่อ **INT** / **RST**

HALLEY V2.1

โมดูล AR VR 9-DOF IMU Sensor

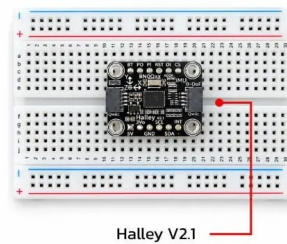


แบบที่ 1 — เสียบ Qwiic



ESP32 → Qwiic → Halley V2.1

แบบที่ 2 — บัดกรีขาแล้วใช้ SPI



ESP32 → Pin Header → Halley V2.1

รูปที่ 2.2 รูปแบบการต่อ: แบบที่ 1 เสียบ Qwiic (I2C) และแบบที่ 2 บัดกรีขาแล้วใช้ SPI

บทที่ 3 การติดตั้งซอฟต์แวร์

3.1 ติดตั้ง Arduino IDE และ ESP32 board package

1. ดาวน์โหลดและติดตั้ง Arduino IDE 2 จาก arduino.cc/en/software
2. เปิด **File** → **Preferences** (macOS: **Arduino IDE** → **Settings**) แล้วใส่ URL https://espressif.github.io/arduino-esp32/package_esp32_index.json ในช่อง **Additional Boards Manager URLs**
3. เปิด **Tools** → **Board** → **Boards Manager...** ค้นหา **esp32** แล้วติดตั้ง **esp32 by Espressif Systems** เวอร์ชัน 3.x ขึ้นไป (ทดสอบกับ 3.3.12)
4. ถ้าใช้ Arduino Nano ไม่ต้องติดตั้งเพิ่ม (Arduino AVR Boards มากับโปรแกรม)

3.2 ติดตั้งไลบรารี Massmore_BNO08x

1. ดาวน์โหลดไฟล์ `ArduinoIDE/Massmore_BNO08x.zip` จาก GitHub ของ Massmore
2. เปิด **Sketch** → **Include Library** → **Add .ZIP Library...** แล้วเลือกไฟล์ ZIP
3. ตรวจสอบ: **File** → **Examples** → **Massmore_BNO08x** ต้องเห็นตัวอย่าง 10 ชุด ตั้งแต่ `01_BasicRead` ถึง `10_Factory_Test`

สำคัญ: ถ้าเคยติดตั้งเวอร์ชันเก่า (1.x หรือ 2.0.x) ให้ลบโฟลเดอร์ `Documents/Arduino/libraries/Massmore_BNO08x` เดิมก่อน API ของเวอร์ชัน 1.x (`MassmoreBNO08x`, `getLastError()`) ใช้ร่วมกับเวอร์ชัน 2.x ไม่ได้

3.3 การตั้งค่าบอร์ดใน Arduino IDE

ตารางที่ 3.1 การตั้งค่าในเมนู Tools ที่ใช้ทดสอบ

รายการ	ESP32 DevKit	ESP32-S3 (MOMO)	Arduino Nano
Board	ESP32 Dev Module	ESP32S3 Dev Module	Arduino Nano
Upload Speed	921600 (ไม่นิ่ง ลด 460800)	921600	—
USB CDC On Boot	—	Disabled	—
Processor	—	—	ATmega328P (หรือ Old Bootloader)
Serial Monitor	115200 baud	115200 baud	115200 baud

ทริป: บอร์ด MOMO (ESP32-S3) ส่ง Serial ผ่านชิป USB-UART บนบอร์ด ต้องตั้ง **USB CDC On Boot = Disabled** มิฉะนั้น Serial Monitor จะไม่แสดงข้อความ
ปิด Serial Monitor ก่อนอัปโหลดทุกครั้ง ถ้าเปิดค้างไว้ โปรแกรมจะเปิดพอร์ตไม่ได้ (`Resource busy`)

3.4 ใช้งานด้วย PlatformIO (VS Code)

1. ติดตั้ง VS Code และส่วนขยาย **PlatformIO IDE**
2. เลือก **File** → **Open Folder...** แล้วเปิดโฟลเดอร์ **PlatformIO/** ของโปรเจกต์
3. เลือก environment ที่แถบล่าง: **esp32-s3-devkitc-1** (ค่าเริ่มต้น), **esp32dev** หรือ **nano**
4. กด **Build** → **Upload** → **Monitor** (Serial 115200)

ไม่ต้องติดตั้งไลบรารีเพิ่ม เพราะไลบรารีถูกรวมไว้ใน

PlatformIO/lib/Massmore_BNO08x/ แล้ว และ **platformio.ini** กำหนด **platform pioarduino 55.03.311** ไว้ให้ได้ **Arduino-ESP32 Core 3.3.11**

ตารางที่ 3.2 Environment ใน platformio.ini

Environment	บอร์ด	หมายเหตุ
esp32-s3-devkitc-1	ESP32-S3 (MOMO)	ค่าเริ่มต้น · - DARDUINO_USB_CDC_ON_BOOT=0
esp32dev	ESP32 DevKit (Classic)	upload 921600 (มีค่าทางเลือก 512000 / 460800 ในคอมเมนต์)
nano	Arduino Nano (ATmega328P)	nanoatmega328new · bootloader แก้ไข nanoatmega328

ไฟล์ **src/main.cpp** เป็นสำเนาของตัวอย่าง **03_GameRotationVector_Euler**
ถ้าต้องการลองตัวอย่างอื่น ให้คัดลอกเนื้อหาไฟล์ **.ino** จาก
ArduinoIDE/Massmore_BNO08x/examples/<ชื่อตัวอย่าง>/ มาทับ **src/main.cpp**
แล้วเพิ่มบรรทัด **#include <Arduino.h>** ไว้บรรทัดแรก

บทที่ 4 เริ่มต้นใช้งานผ่าน I2C

บทนี้เป็นจุดเริ่มต้นที่แนะนำสำหรับผู้ใช้งานทุกคน ใช้โหมด I2C (ค่าเริ่มต้นจากโรงงาน ไม่ต้องตั้ง P0 / P1) และตัวอย่าง 01_BasicRead ซึ่งเรียก `readAll()` เพียงครั้งเดียวก็ได้ quaternion, roll / pitch / yaw, accelerometer, gyroscope และ magnetometer ครบทุกค่า

4.1 อุปกรณ์ที่ต้องเตรียม

ตารางที่ 4.1 รายการอุปกรณ์

รายการ	จำนวน	หมายเหตุ
บอร์ด Massmore Halley V2.1 (BNO085 / BNO086)	1	—
บอร์ด ESP32 DevKit, ESP32-S3 (MOMO) หรือ Arduino Nano	1	ตัวอย่างหลักใช้ ESP32 DevKit
สาย USB สำหรับรับส่งข้อมูล	1	สายชาร์จอย่างเดียวอัปโหลดไม่ได้
สาย Qwiic หรือสายจัมเปอร์	1 ชุด	3Vo, GND, SDA, SCL และ RST (แนะนำ)

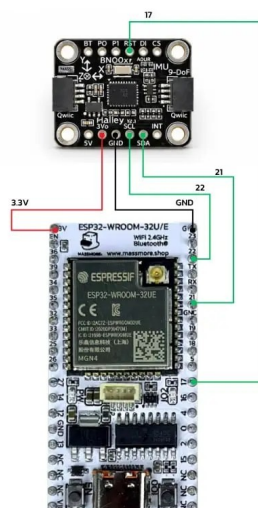
4.2 การต่อวงจร

HALLEY V2.1

โมดูล AR VR 9-DOF IMU Sensor



- Power
- I²C
- GND



รูปที่ 4.1 การต่อ I2C ระหว่าง Halley V2.1 กับ ESP32 DevKit

ตารางที่ 4.2 การต่อสาย I2C

Halley V2.1	ESP32 DevKit	ESP32-S3 (MOMO)	Arduino Nano	จำเป็น
3Vo (หรือ 5V)	3V3	3V3	5V → pad 5V	✓
GND	GND	GND	GND	✓
SDA	GPIO 21	GPIO 14	A4	✓
SCL	GPIO 22	GPIO 15	A5	✓
RST	GPIO 17	GPIO 18 (เปิดคอมเมนต์)	—	แนะนำ
INT	GPIO 4 (ตัวอย่าง 02, 04)	—	D2	ไม่บังคับ
P0, P1, DI, BT	ไม่ต่อ	ไม่ต่อ	ไม่ต่อ	—

ทำไมควรต่อขา RST

เมื่อ MCU ถูกรีเซ็ตกลางการรับส่งข้อมูล I2C เช่น ตอนอัปโหลดโปรแกรมใหม่หรือตอนเปิด Serial Monitor ขีปอาจกดสาย SDA (หรือทั้ง SDA และ SCL) ค้างไว้ที่ระดับ LOW ทำให้ `begin()` หาขีปไม่พบ อาการนี้พบบ่อยขึ้นเมื่ออัตรา report สูง ถ้าต่อ RST เข้า GPIO แล้วส่งหมายเลขขาให้ `begin()` ไบรารีจะรีเซ็ตขีปก่อนใช้งานทุกครั้ง ผลทดสอบรีเซ็ตกลางการส่งข้อมูล 100 Hz จำนวน 5 ครั้ง ขีปกลับมาทำงานได้ครบ 5 ครั้ง ถ้าไม่ได้ต่อ RST ให้แก้ `#define RST_PIN 17` เป็น `-1` และเมื่อพบข้อความ `Bus I/O error` ให้ถอดไฟเซนเซอร์ประมาณ 5 วินาทีแล้วเสียบใหม่

ตรวจก่อนเสียบสาย USB

- ☐ 3Vo ต่อ 3V3 (หรือ 5V ต่อ 5V) ไม่สลับกัน
- ☐ GND ต่อร่วมกัน
- ☐ SDA / SCL ไม่สลับ
- ☐ หมายเลขขาในโค้ดตรงกับขาที่ต่อจริง
- ☐ BT ไม่ได้ต่อลง GND

4.3 อัปโหลดตัวอย่าง 01_BasicRead

- เปิด File → Examples → Massmore_BNO08x → 01_BasicRead
- ตัวอย่างเลือกขาให้เองตามชนิด MCU (ESP32: SDA 21 / SCL 22 / RST 17 · ESP32-S3: SDA 14 / SCL 15 · Nano: A4 / A5) ไม่ต้องแก้ถ้าต่อตามตารางที่ 4.2
- เลือก Board และ Port ตามตารางที่ 3.1
- กด Upload ถ้าค้างที่ Connecting... ให้กดปุ่ม BOOT ค้างไว้จนเริ่มเขียน flash
- เปิด Serial Monitor ที่ 115200 baud

ส่วนสำคัญของโค้ด

```
#include <Wire.h>
#include <Massmore_BNO08x.h>

Massmore_BNO08x imu;

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22); // ESP32 กำหนดขาเอง (Nano: Wire.begin());
  Wire.setClock(100000); // 100 kHz เสถียรที่สุด
  if (!imu.begin(0x4A, Wire, -1, 17)) { // address, bus, INT, RST
    Serial.println(Massmore_BNO08x::statusToString(imu.lastError()));
    while (true) delay(100);
  }
}

void loop() {
  Massmore_BNO08x_reading_t r;
  if (imu.readAll(r)) { // Blocking: ได้ทุกค่าในครั้งเดียว
    Serial.print("roll="); Serial.print(r.eulerDeg.roll, 1);
    Serial.print(" pitch="); Serial.print(r.eulerDeg.pitch, 1);
    Serial.print(" yaw="); Serial.println(r.eulerDeg.yaw, 1);
  }
  delay(100);
}
```

ผลลัพธ์ที่ควรเห็น

ผลจริงจากการทดสอบบน ESP32 DevKit + BNO086

```
Massmore_BNO08x - 01_BasicRead
BNO08x connected at 0x4A
heading=359.9 roll=-1.8 pitch=1.7 yaw=-0.1
| accel=-0.29 -0.30 9.58 gyro=0.00 0.00 0.00 mag=36.2 -21.6 -32.7
[RV acc: Unreliable]
```

ตารางที่ 4.3 ความหมายของค่าที่แสดง

ค่า	หน่วย	ความหมาย
roll / pitch / yaw	องศา (−180..180)	มุมเอียงรอบแกน X / Y / Z จาก Rotation Vector (9 แกน)
heading	องศา (0..360)	yaw ที่แปลงเป็นช่วง 0..360 (นับทวนเข็มนาฬิกา) — ถ้าต้องการทิศแบบเข็มทิศ ดูบทที่ 8
accel	m/s ²	ความเร่งรวมแรงโน้มถ่วง วางราบหนึ่งแกน z ≈ 9.6–9.8
gyro	rad/s	ความเร็วเชิงมุม วางนิ่ง ≈ 0
mag	μT	สนามแม่เหล็ก ขนาดรวมโดยทั่วไป 25–65 μT
RV acc	Unreliable / Low / Medium / High	ความน่าเชื่อถือของทิศทางที่ชิปประเมิน

เกณฑ์ผ่าน

- มีข้อความ **BNO08x connected at 0x4A**
- วางบอร์ดราบนิ่ง accel แกน z อยู่ระหว่าง 9.6–9.8 และ gyro ใกล้ 0
- เอียงบอร์ดแล้ว roll / pitch เปลี่ยนตามทิศที่เอียง

หมายเหตุ: RV acc: Unreliable เป็นเรื่องปกติก่อน calibrate magnetometer ค่า roll / pitch ใช้งานได้ทันที ส่วน heading ที่อ้างอิงทิศเหนือต้อง calibrate ก่อน (บทที่ 7) หรือถ้าไม่ต้องการทิศเหนือ ให้ใช้ Game Rotation Vector (บทที่ 5)

ทริป: ถ้าขึ้น **BNO08x not found: No device found** หรือ **Bus I/O error** ดูหัวข้อ 13.1 ในบทการแก้ไขปัญหา

4.4 ย้ายขาและใช้ I2C bus ที่สอง (ตัวอย่าง 02)

ESP32 และ ESP32-S3 ย้ายสัญญาณ I2C ไปขาใดก็ได้ผ่าน GPIO Matrix และมี I2C สองชุด (**Wire**, **Wire1**) ไบรารีไม่กำหนดขาเอง sketch เป็นผู้เรียก **begin()** ของ bus แล้วส่ง reference เข้ามา ตัวอย่าง **02_CustomPins_BusRemap** ใช้ **Wire1** และอ่านเฉพาะ Euler angles ด้วย **readEulerDeg()**

```
Wire1.begin(21, 22, 100000); // SDA, SCL, ความถี่ (Core 3.x)
imu.begin(0x4A, Wire1, /*INT*/ 4, /*RST*/ 17);

Massmore_BNO08x_euler_t e;
if (imu.readEulerDeg(e)) { /* e.roll, e.pitch, e.yaw */ }
```

ผลจริงบน ESP32 DevKit

```
Massmore_BNO08x - 02_CustomPins_BusRemap
Wire1 on SDA=21 SCL=22
BNO08x connected at 0x4A
roll=-1.8 pitch=1.7 yaw=-0.1
```

Arduino Nano มี I2C ชุดเดียวที่ขา A4 / A5 ตายตัว ตัวอย่างนี้จึงใช้ **Wire** ตามปกติและทำงานเหมือนตัวอย่าง 01

บทที่ 5 วัดมุมโดยไม่ต้อง calibrate

ตัวอย่าง `03_GameRotationVector_Euler` (ใหม่ในเวอร์ชัน 2.1.1) อ่าน roll / pitch / yaw ด้วย **Game Rotation Vector** ซึ่งเป็น fusion 6 แกน (accelerometer + gyroscope) ไม่ใช้ magnetometer จึงไม่ต้อง calibrate เปิดเครื่องแล้ววัดได้ทันที และไม่ได้รับผลจากโลหะ มอเตอร์ หรือสายไฟรอบตัว เหมาะกับงานวัดมุมเอียง gimbal และงานที่อยู่ใกล้มอเตอร์ ตัวอย่างนี้เป็นโปรแกรมเริ่มต้นของโปรเจกต์ PlatformIO ด้วย

5.1 ความต่างจาก Rotation Vector

ตารางที่ 5.1 Game Rotation Vector เทียบกับ Rotation Vector

หัวข้อ	Game Rotation Vector (ตัวอย่าง 03)	Rotation Vector (ตัวอย่าง 05, 06)
เซนเซอร์ที่ใช้	Accel + Gyro	Accel + Gyro + Mag
ต้อง calibrate	ไม่ต้อง	ต้อง calibrate magnetometer
yaw = 0 คือ	ทิศที่บอร์ดหันอยู่ตอนเปิดเครื่อง	อ้างอิงทิศเหนือแม่เหล็ก
yaw drift	drift ช้า ๆ (วางนิ่ง 5 นาที $\approx 0.07^\circ$ จากการทดสอบ)	ไม่ drift เมื่อ calibrate แล้ว
roll / pitch	ไม่ drift	ไม่ drift
ผลจากสนามแม่เหล็ก	ไม่มี	มี

5.2 การต่อวงจรและการใช้งาน

ต่อวงจรเหมือนตัวอย่าง 01 (ตารางที่ 4.2) ตัวอย่างนี้ไม่บังคับต่อ RST ถ้าต่อไว้ให้เปิดคอมเมนต์บรรทัด `#define RST_PIN` ถ้าไม่ได้ต่อ ตัวอย่างจะสั่ง `softReset()` ให้เพื่อล้าง report ที่โปรแกรมก่อนหน้านี้เปิดค้างไว้

ตารางที่ 5.2 คำสั่งทาง Serial Monitor (ส่งทีละตัวอักษร)

คำสั่ง	การทำงาน
z	ตั้งศูนย์เฉพาะ yaw — วางบอร์ดในท่าที่ต้องการให้เป็น 0 แล้วกด (หัก offset ในซอฟต์แวร์)
a	ตั้งศูนย์ทุกแกน — วางบอร์ดให้ได้ระดับก่อนกด แล้ว roll / pitch / yaw จะเป็น 0 ทั้งหมด
r	ยกเลิก tare กลับไปใช้ท่าตอนเปิดเครื่องเป็นศูนย์

รูปแบบผลลัพธ์ (ตัวเลขขึ้นกับท่าวางบอร์ด)

```
Massmore_BNO08x - 03_GameRotationVector_Euler (no calibration needed)
BNO08x connected at 0x4A
z = zero yaw  a = zero all axes  r = undo tare
roll= -1.32 pitch= 1.75 yaw= 0.02 deg
> all axes zeroed
roll= 0.00 pitch= 0.00 yaw= 0.00 deg
```

หมายเหตุ: tare ของตัวอย่างนี้เก็บใน RAM ของชิป หายเมื่อตัดไฟ ไม่เขียน flash จึงกดได้บ่อยตามต้องการ

เมื่อใช้ Game RV ต้องสั่ง `tareNow(..., MASSMORE_BNO08X_TARE_BASIS_GAMING_RV)` ให้ basis ตรงกับ report ที่เปิด และการ tare เฉพาะแกน Z ด้วย basis นี้ไม่มีผล (ทดสอบบน BNO086 FW 3.12.6) ตัวอย่างจึงหัก offset ของ yaw ในซอฟต์แวร์แทน

ข้อควรระวัง: อย่าเปิด Rotation Vector, Game RV และ Geomagnetic RV พร้อมกัน ทุก report เขียนค่า quaternion ชุดเดียวกันในไลบรารี มุมที่อ่านได้จะสลับไปตาม report ที่มาถึงทีหลัง

บทที่ 6

Non-blocking API สำหรับงานหลายภารกิจ

`readAll()` ใช้ง่ายแต่จะรอนจนได้ข้อมูล (blocking) ในงานที่ MCU ต้องทำหลายอย่างพร้อมกัน เช่น ควบคุมมอเตอร์ อ่านปุ่ม หรือสื่อสาร Wi-Fi ควรใช้ Non-blocking FSM API ซึ่งไม่หยุดรอ ตัวอย่าง [04_NonBlocking_Multitask](#) เปิด Rotation Vector และ Accelerometer ที่ 100 Hz พร้อมกะพริบ LED ทุก 500 ms และนับจำนวนรอบ `loop()` ต่อวินาที

6.1 ลำดับการทำงานของ FSM API

ตารางที่ 6.1 ขั้นตอนของ FSM API

ขั้น	ฟังก์ชัน	หน้าที่
1	<code>enableRotationVector(us)</code> ฯลฯ	เปิด report ที่ต้องการใน <code>setup()</code> — argument คือ คาบเวลาหน่วยไมโครวินาที
2	<code>update()</code> / <code>updateAll()</code>	เรียกใน <code>loop()</code> ทุกรอบ ดึง packet ที่ค้างอยู่ ถ้าไม่มีจะคืนทันที
3	<code>isDataReady(id)</code> / <code>hasNewReport(id)</code>	ตรวจว่ามี report ใหม่ (<code>hasNewReport</code> ล้าง flag ให้ด้วย)
4	<code>getReadings(r)</code> หรือ getter อื่น	คัดลอกค่าล่าสุดจากหน่วยความจำ ไม่ใช่ bus

```
void setup() {
  // ... Wire.begin(), imu.begin() ...
  imu.enableRotationVector(MASSMORE_BNO08X_INTERVAL_100HZ);
  imu.enableAccelerometer(MASSMORE_BNO08X_INTERVAL_100HZ);
}

void loop() {
  imu.update(); // ไม่รอ
  if (imu.hasNewReport(MASSMORE_BNO08X_SENSOR_ROTATION_VECTOR)) {
    Massmore_BNO08x_reading_t r;
    imu.getReadings(r); // ไม่แตะ bus
  }
  // งานอื่นทำได้ทันที — จับเวลาด้วย (uint32_t)(millis() - t0) >= period
}
```

6.2 อัตรา report

ตารางที่ 6.2 ค่าคงที่ของอัตรา report

ค่าคงที่	คาบเวลา (μs)	อัตรา
<code>MASSMORE_BNO08X_INTERVAL_1HZ</code>	1,000,000	1 Hz
<code>MASSMORE_BNO08X_INTERVAL_10HZ</code>	100,000	10 Hz
<code>MASSMORE_BNO08X_INTERVAL_50HZ</code>	20,000	50 Hz
<code>MASSMORE_BNO08X_INTERVAL_100HZ</code>	10,000	100 Hz

ค่าคงที่	คาบเวลา (μs)	อัตรา
MASSMORE_BNO08X_INTERVAL_200HZ	5,000	200 Hz
MASSMORE_BNO08X_INTERVAL_400HZ	2,500	400 Hz
MASSMORE_BNO08X_INTERVAL_1000HZ	1,000	1000 Hz

ผ่าน I2C แนะนำไม่เกิน 100–200 Hz ต่อ report ถ้าต้องการอัตราสูงกว่านี้ให้ใช้ SPI (บทที่ 9)

6.3 ผลลัพธ์และการอ่านค่าสถิติ

ผลจบบน ESP32 DevKit (ต่อ INT)

```
Massmore_BNO08x - 04_NonBlocking_Multitask
Rotation Vector + Accelerometer @ 100 Hz. loop() never blocks.
yaw=0.0 pitch=0.0 roll=0.0 | az=0.00          <- บรรทัดแรกยังไม่มี report — ปกติ
yaw=-0.1 pitch=1.9 roll=-1.5 | az=9.57
[stats] loop/s=20427 RV reports/s=38          <- วินาทีแรกยังไม่เต็ม — ปกติ
[stats] loop/s=36811 RV reports/s=100
[stats] loop/s=36312 RV reports/s=100
```

- **RV reports/s** ≈ 100 แสดงว่าได้ report ครบตามอัตราที่ตั้ง
- **loop/s** สูงระดับหลักหมื่น แสดงว่า **loop()** ไม่ถูก block (ESP32-S3 ที่ไม่ต่อ INT วัดได้ราว 640 รอบต่อวินาที เพราะต้อง poll bus เอง แต่ยังได้ RV 100 Hz ครบ)
- ต่อขา **INT** จะช่วยให้ไลบรารีอ่าน bus เฉพาะเมื่อชิปมีข้อมูล

บทที่ 7

Calibration และ Tare

ตัวอย่าง **05_Calibration_Tare** ทำ calibration ตามขั้นตอนของผู้ผลิตชิป (เอกสาร 1000-4044) และ tare ตามเอกสาร 1000-4045 ควบคุมทาง Serial Monitor (115200) โดยส่งคำสั่งทีละตัวอักษร ท้ายบรรทัดสถานะจะบอกเองว่าต้องทำอะไรต่อ

7.1 Calibration กับ Tare ต่างกันอย่างไร

- **Calibration** คือการให้ชิปเรียนรู้ค่าชดเชยของเซนเซอร์ (bias, soft/hard iron ของ magnetometer) เพื่อให้ทิศทางถูกต้อง โดยเฉพาะ heading
- **Tare** คือการกำหนดว่าค่าปัจจุบันเป็นศูนย์ เช่น ให้ทิศทางของหุ่นยนต์เป็น yaw = 0 ต้องทำหลัง calibration

7.2 ขั้นตอน calibration

1. อัปโหลด **05_Calibration_Tare** แล้วเปิด Serial Monitor 115200
2. กด **c** เพื่อเปิด dynamic calibration ของ accelerometer, gyroscope และ magnetometer (หรือ **m** เฉพาะ magnetometer)
3. ขยับบอร์ดตามท่าในตารางที่ 7.1 จนท้ายบรรทัดสถานะขึ้น **READY - press 's' to save**
4. กด **s** เพื่อบันทึกค่า calibration (DCD) ลง flash ของชิป ค่าจะคงอยู่หลังตัดไฟ
5. กด **e** เพื่อปิด dynamic calibration

ตารางที่ 7.1 ท่าขยับบอร์ดตาม datasheet

เซนเซอร์	ท่าทาง	เวลา
Magnetometer	หมุนประมาณ 180° แล้วหมุนกลับ รอบแกน roll, pitch และ yaw ทีละแกน	≈ 2 วินาทีต่อแกน
Accelerometer	ถือนิ่งในทิศทางที่ต่างกันชัดเจน 4-6 ทิศ	≈ 1 วินาทีต่อทิศ
Gyroscope	วางนิ่งบนพื้นที่มั่นคง	2-3 วินาที

หมายเหตุ: ท่าแกว่งเป็นเลข 8 ที่พบในคู่มือเซนเซอร์รุ่นอื่น (รวมถึงข้อความแนะนำในตัวอย่าง 06) ก็ช่วยให้ magnetometer เรียนรู้ได้ แต่ขั้นตอนที่ผู้ผลิต BNO08x กำหนดคือการหมุนไป-กลับรอบทีละแกนตามตารางที่ 7.1

บรรทัดสถานะ

รูปแบบบรรทัดสถานะ (พิมพ์ทุก 500 ms)

```
roll=0.0 pitch=0.0 yaw=0.0 | mag=Unreliable RV=Unreliable err=0.0 deg
| press 'c' to start calibration
> calibration ON (accel+gyro+mag) - now move the board
roll=1.2 pitch=-0.8 yaw=35.4 | mag=Medium RV=Medium err=15.9 deg
| MOVE THE BOARD (press 'h' for the pattern)
roll=0.9 pitch=-0.5 yaw=36.1 | mag=High RV=High err=2.9 deg
| READY - press 's' to save
```

err คือค่าคลาดเคลื่อนของ heading ที่ชิปประเมิน (องศา) ตัวอย่างถือว่าพร้อมบันทึกเมื่อ err ต่ำกว่า 10° ตามคำแนะนำของผู้ผลิต ผลทดสอบจริงบน ESP32-S3: คำสั่ง ? ตอบ ME calibration status = 0 (OK) และคำสั่ง z ปรับ yaw จาก 147.2° เป็น -0.8°

7.3 คำสั่งทั้งหมด

ตารางที่ 7.2 คำสั่งของตัวอย่าง 05

กลุ่ม	คำสั่ง	การทำงาน
Calibration	c / m	เปิด calibration ทั้งหมด / เฉพาะ magnetometer
	e	ปิด dynamic calibration
	s	บันทึก calibration (Save DCD) ลง flash
	x	ลบ calibration แล้วรีเซ็ตชิป
	?	ถามสถานะ calibration
Tare	z	ตั้งศูนย์เฉพาะ heading (แกน Z)
	a	ตั้งศูนย์ทุกแกน (วางระดับและหันไปทิศอ้างอิงก่อน)
	p	บันทึก tare ลง flash ให้คงอยู่หลังตัดไฟ
	t	ล้าง tare ที่บันทึกไว้ (clearTare())
อื่น ๆ	h	แสดงคำสั่งอีกครั้ง

ข้อควรระวัง: คำสั่ง s, p, x, t เขียน flash ของชิปซึ่งมีจำนวนรอบการเขียนจำกัด ใช้ตอนติดตั้งหรือตั้งค่าเครื่องเท่านั้น ห้ามเรียกซ้ำใน loop()

softReset() ไม่ล้าง tare (ยืนยันจากการทดสอบ) ต้องใช้ clearTare()

7.4 สภาพแวดล้อมที่เหมาะสม

- ทำ calibration หลังติดตั้งเซนเซอร์ในกล่องหรือโครงจริงแล้ว
- จัดตำแหน่งมอเตอร์และสายกระแสน้ำสูงให้เหมือนตอนใช้งานจริง
- หลีกเลียงโต๊ะเหล็ก แม่เหล็ก ลำโพง และไขควงที่มีแม่เหล็ก
- ถ้า heading ยังไม่นิ่ง ให้ลองเทียบกับ Game Rotation Vector (บทที่ 5) เพื่อแยกปัญหาสนามแม่เหล็กออกจากปัญหาการสื่อสาร

บทที่ 8

Euler angles และเข็มทิศ

ตัวอย่าง `06_I2C_Euler_Compass` อ่าน Rotation Vector (9 แกน) แล้วแสดง roll / pitch / yaw พร้อมเข็มทิศแบบชดเชยการเอียง (tilt-compensated) $0..360^\circ$ ชื่อทิศ 16 ทิศ (N, NNE, NE, ...) และค่าความแม่นยำของ heading ที่ชิปประเมิน ชิปทำ tilt compensation ให้แล้ว จึงเอียงบอร์ดได้โดย heading ไม่เพี้ยน

8.1 แกนอ้างอิงและการคำนวณ heading

Rotation Vector อ้างอิงกรอบพิกัดโลกแบบ East-North-Up โดย yaw = 0 เมื่อแกน +X ของบอร์ดชี้ทิศตะวันออก และเพิ่มขึ้นทวนเข็มนาฬิกา ส่วนเข็มทิศนับ 0 = ทิศเหนือ เพิ่มตามเข็มนาฬิกา ตัวอย่างจึงคำนวณ heading ของแกน +X ดังนี้

```
heading = wrap360(90 - yaw + DECLINATION_DEG + HEADING_OFFSET_DEG)
```

- `DECLINATION_DEG` ค่ามุมเบี่ยงเบนแม่เหล็ก (ตะวันออก = +) ประเทศไทยใกล้ 0° (กรุงเทพฯ $\approx -0.5^\circ$)
- `HEADING_OFFSET_DEG` มุมติดตั้งบอร์ดเทียบกับทิศหน้าของอุปกรณ์ ถ้าติดให้ +X ชี้ไปข้างหน้าใช้ 0
- ค่า `headingDeg` ของ `readAll()` และ `getHeadingDeg()` คือ yaw ในช่วง $0..360$ แบบทวนเข็มนาฬิกา ไม่ใช่ทิศแบบเข็มทิศ ถ้าต้องการทิศเหนือให้ใช้สูตรข้างต้น

8.2 ผลลัพธ์

รูปแบบผลลัพธ์ (พิมพ์ 5 Hz)

```
Massmore_BNO08x - 06_I2C_Euler_Compass
BNO08x connected at 0x4A
Move the board in a figure-8 until RV accuracy is MEDIUM or HIGH.
roll=    0.4 pitch=  -1.2 yaw=   12.6
  | heading= 77.4 ENE   +/-4 deg [RV High, MAG High]
```

ถ้า accuracy ของ RV ต่ำกว่า Medium ตัวอย่างจะเตือนทุก 5 วินาทีให้ calibrate ให้ทำตามขั้นตอนในบทที่ 7 แล้วกด s ในตัวอย่าง 05 เพื่อบันทึกค่าไว้ ครั้งต่อไปจะได้ heading ที่แม่นยำเร็วขึ้น

ทริป: heading จากเซนเซอร์แม่เหล็กไวต่อการรบกวนในอาคาร ควรทดสอบในตำแหน่งติดตั้งจริงและห่างจากโครงเหล็กอย่างน้อย 30 cm ถ้างานไม่ต้องการทิศเหนือจริงให้ใช้ Game RV ในบทที่ 5

บทที่ 9

SPI สำหรับงานอัตราสูง

SPI เหมาะกับงานที่ต้องการข้อมูลถี่ เพราะไม่มี address phase ไม่มี clock stretching และย้ายข้อมูลทั้ง packet ได้ในการเลือกชิปครั้งเดียว BNO08x ใช้ SPI Mode 3 (CPOL = 1, CPHA = 1) ความเร็วสูงสุด 3 MHz ตัวอย่าง `07_SPI_Advance` เปิด Rotation Vector 400 Hz, Accelerometer และ Gyroscope 200 Hz และวัดอัตราจริงทุกวินาที

9.1 การต่อวงจร

ตารางที่ 9.1 การต่อสาย SPI

Halley V2.1	ESP32 DevKit	ESP32-S3	Arduino Nano*	หมายเหตุ
3Vo	3V3	3V3	3V3	
GND	GND	GND	GND	
SCL (SCK)	GPIO 18	GPIO 12	D13	
SDA (MISO)	GPIO 19	GPIO 13	D12	
DI (MOSI)	GPIO 23	GPIO 11	D11	
CS	GPIO 5	GPIO 10	D10	
INT	GPIO 4	GPIO 4	D2	จำเป็น
RST	GPIO 17	GPIO 5	D3	จำเป็น
P0 (WAKE)	GPIO 16	GPIO 6	D4	จำเป็น — ห้ามต่อ 3Vo
P1	3Vo	3Vo	3Vo	HIGH
BT	ปล่อยลอย	ปล่อยลอย	ปล่อยลอย	

* ขา DI, CS, INT, RST, P0, P1 เป็นลอจิก 3.3 V Arduino Nano ต้องต่อผ่านวงจรแปลงระดับแรงดัน ขาของ ESP32-S3 ในตารางผ่านการ compile แล้วแต่ยังไม่ได้ทดสอบกับบอร์ดจริง

ข้อควรระวัง: ต้องต่อ P0 เข้า GPIO แล้วส่งเป็น wakePin ห้ามต่อ 3Vo ตรง ๆ และห้ามบัดกรีปิด jumper PS0 ในโหมด SPI ชิปจะรับคำสั่งทาง SPI เฉพาะเมื่อถูกปลุกด้วยขา WAKE (P0 = LOW) ถ้าต่อ 3Vo ไว้ จะอ่าน advertisement ได้แต่ส่งคำสั่งไม่ได้ และขึ้น No device found โลบารี่ตั้ง P0 = HIGH ระหว่างรีเซ็ตให้ชิปเข้าโหมด SPI แล้วใช้ขาเดียวกันเป็น WAKE ต่อ ถ้าไม่ส่ง wakePin ฟังก์ชัน beginSPI() จะคืน ERR_BAD_PARAM ทันที

9.2 โค้ดและผลลัพธ์

```
#include <SPI.h>
#include <Massmore_BNO08x.h>
Massmore_BNO08x imu;

void setup() {
  Serial.begin(115200);
  SPI.begin(18, 19, 23, 5); // SCK, MISO, MOSI, CS
  if (!imu.beginSPI(5, 4, 17, 16, SPI, 3000000)) { // CS INT RST WAKE bus Hz
    Serial.println(Massmore_BNO08x::statusToString(imu.lastError()));
    while (true) delay(100);
  }
  imu.enableRotationVector(MASSMORE_BNO08X_INTERVAL_400HZ);
}
void loop() { imu.updateAll(32); /* ... */ }
```

ผลจบบน ESP32 DevKit

```
Massmore_BNO08x - 07_SPI_Advance
Connected. Firmware 3.12.6 part 10004563
rpy -1.1 1.9 -0.0 gyro(dps) 0.0 0.0 0.0
[rate] Rotation Vector = 130 Hz      <- วินาทีแรกยังไม่เต็ม — ปกติ
[rate] Rotation Vector = 400 Hz
[rate] Rotation Vector = 397 Hz
[rate] Rotation Vector = 399 Hz
```

อัตราที่วัดได้ต่อเนื่อง 15 วินาทีอยู่ระหว่าง 395–400 Hz และรีเซ็ตซ้ำแล้วกลับมาทำงานได้ทุกครั้ง ถ้าสายยาว ให้ลดความเร็ว SPI เป็น 1 MHz (`SPI_SPEED_HZ 1000000UL`)

บทที่ 10

การเชื่อมต่อแบบ UART

BNO08x มีโหมด UART สองแบบที่ต่างกันมาก เลือกตามความต้องการของงาน ด้วยตารางที่ 10.1

ตารางที่ 10.1 เปรียบเทียบ UART-SHTP และ UART-RVC

หัวข้อ	UART-SHTP (ตัวอย่าง 09)	UART-RVC (ตัวอย่าง 08)
P1 / P0	1 / 0 (บนบอร์ดเขียน UART)	0 / 1 (บนบอร์ดเขียน UART+)
ความเร็ว	3,000,000 baud 8N1	115200 baud 8N1
สายข้อมูล	2 เส้น (TX, RX)	1 เส้น (TX ของชิป)
ข้อมูลที่ได้	API เต็มรูปแบบเหมือน I2C	yaw / pitch / roll (0.01°) และ accel 3 แกน ที่ 100 Hz
สั่งงานชิป	ได้ (calibration, tare, Product ID ฯลฯ)	ไม่ได้ ชิปส่งข้อมูลเองตลอดเวลา
MCU ที่รองรับ	ESP32 / ESP32-S3 เท่านั้น	ESP32 / ESP32-S3 / Nano (SoftwareSerial)

สำคัญ: ในโหมด UART ทั้งสองแบบ pad SDA คือ TX ของชิป ต้องต่อเข้า RX ของ MCU และ pad SCL คือ RX ของชิป ต่อเข้า TX ของ MCU การสลับ TX / RX เป็นสาเหตุที่พบบ่อยที่สุด

10.1 UART-RVC (ตัวอย่าง 08)

โหมดที่ง่ายที่สุด: ตั้งขาสองขา ต่อสายข้อมูลเส้นเดียว แล้วชิปจะส่ง yaw / pitch / roll และความเร่งที่ 100 Hz ตลอดเวลาโดย host ไม่ต้องส่งคำสั่ง เหมาะกับหุ่นยนต์ที่ต้องการเพียง heading ใช้คลาส `Massmore_BNO08x_RVC` จากไฟล์

`Massmore_BNO08x_RVC.h`

ตารางที่ 10.2 การต่อสาย UART-RVC

Halley V2.1	ESP32 DevKit	ESP32-S3 (MOMO)	Arduino Nano*
3Vo / GND	3V3 / GND	3V3 / GND	3V3 / GND
SDA (TX ของชิป)	GPIO 21 (RX)	GPIO 14 (RX)	D2 (SoftwareSerial RX)
SCL	GPIO 22 (ไม่ใช่)	GPIO 15 (ไม่ใช่)	—
RST	GPIO 17 (แนะนำ)	—	—
P1 / P0	GND / 3Vo	GND / 3Vo	GND / 3Vo

* Nano ต้องผ่านวงจรแปลงระดับแรงดัน ถ้าไม่ได้ต่อ RST ให้ถอดไฟเซนเซอร์หลังเปลี่ยน P0 / P1 ตัวอย่างจะส่งพัลส์ RST หนึ่งครั้งตอนเริ่ม เพื่อให้ชิปเข้าโหมด RVC

ผลจริงบน ESP32 DevKit (พิมพ์ 10 Hz จากข้อมูล 100 Hz)

```
Massmore_BNO08x - 08_UART_RVC
yaw  pitch  roll  ax  ay  az
0.00  -0.93  1.53  -0.24  -0.16  9.68
0.00  -0.92  1.52  -0.22  -0.14  9.65
0.06  -0.84  1.39  -0.24  -0.13  9.63
```

ถ้ามี frame หายหรือ checksum ผิดท้ายบรรทัดจะแสดง (dropped n, bad csum m)
ผลทดสอบ 30 วินาทีไม่พบข้อความนี้เลย

10.2 UART-SHTP (ตัวอย่าง 09)

ใช้ API เดียวกับ I2C ทุกประการ (`readAll()`, `enable*()`, calibration, tare, `isGenuine()`) เพียงเปลี่ยน `begin()` เป็น `beginUART()`

ตารางที่ 10.3 การต่อสาย UART-SHTP

Halley V2.1	ESP32 DevKit	ESP32-S3 (MOMO)
3Vo / GND	3V3 / GND	3V3 / GND
SDA (TX ของชิป)	GPIO 21 (RX)	GPIO 14 (RX)
SCL (RX ของชิป)	GPIO 22 (TX)	GPIO 15 (TX)
INT	GPIO 4 (ไม่บังคับ)	—
RST	GPIO 17 (แนะนำ)	—
P1 / P0	3Vo / GND	3Vo / GND

```
Serial2.setRxBufferSize(1024); // ต้องเรียกก่อน begin()
Serial2.begin(3000000, SERIAL_8N1, 21, 22); // RX = 21, TX = 22
if (!imu.beginUART(Serial2, /*INT*/ 4, /*RST*/ 17)) { /* error */ }
```

ผลจริงบน ESP32 DevKit — 60 วินาที สำเร็จ 469 / 469 ครั้ง

```
Massmore_BNO08x - 09_UART_Mode
UART OK - BNO086 FW 3.12.6 part 10004563
heading=360.0 roll=-1.1 pitch=1.3
| accel=-0.20 -0.17 9.63 gyro=-0.00 0.00 0.00 mag=42.2 2.6 -31.9
[RV acc: Unreliable]
```

- ความเร็วต้องเป็น **3,000,000 baud** ตาม datasheet ค่าอื่นใช้ไม่ได้
- ต้องขยาย RX buffer เป็น 1024 byte ก่อน `begin()` เพราะข้อมูลประกาศ (advertisement) ตอนรีเซ็ตยาวประมาณ 290 byte
- ห้ามต่อผ่านวงจรแปลงระดับแรงดันแบบ pull-up เพราะขอบสัญญาณไม่ทันความเร็ว 3 Mbaud ให้ต่อตรงกับ MCU 3.3 V
- Arduino Nano ทำความเร็วนี้ไม่ได้ ตัวอย่างจะพิมพ์แจ้งให้ใช้ตัวอย่าง 08 แทน

บทที่ 11

การตรวจสอบของแท่งและ Factory Test

BNO08x ไม่มีรีจิสเตอร์ WHO AM I แบบเซนเซอร์ทั่วไป ไบรารีจึงตรวจตัวตนชิปจากข้อมูล Product ID ของเฟิร์มแวร์ SH-2 ร่วมกับ metadata ของ MotionEngine ตัวอย่าง **10_Factory_Test** รวมการตรวจทั้งหมดเป็นชุดทดสอบสำหรับการตรวจรับสินค้า (Outgoing QA) และพิมพ์ผลในรูปแบบที่ Massmore Web Serial Monitor อ่านได้อัตโนมัติ

11.1 ฟังก์ชันตรวจสอบตัวตนชิป

ตารางที่ 11.1 ฟังก์ชันด้านตัวตนและความเป็นของแท่ง

ฟังก์ชัน	สิ่งที่ตรวจ	ค่าที่คืน
<code>verifyChipID()</code>	part number ใน Product ID (report 0xF8) ตรงกับเฟิร์มแวร์ SH-2 ที่รู้จัก (10003606 / 10004095 / 10004563)	<code>bool</code>
<code>getChipModel()</code>	แยก BNO085 / BNO086 จาก part number และ report เฉพาะรุ่น	<code>chip_t</code>
<code>getSerialNumber()</code>	serial number จาก FRS record 0x4B4B (0 = ไม่ได้บันทึกจากโรงงาน)	<code>uint32_t</code>
<code>verifyMotionEngine()</code>	metadata ของ Rotation Vector มี Q point = 14 / 12	<code>bool</code>
<code>isGenuine()</code>	รวมผล Product ID, version, part number และ metadata	<code>bool</code>
<code>verifyChip()</code>	ผลละเอียด เช่น <code>AUTH_OK</code> , <code>AUTH_UNKNOWN_FW</code> , <code>AUTH_NO_RESPONSE</code>	<code>auth_t</code>

หมายเหตุ: การตรวจนี้เป็น การตรวจความถูกต้องระดับโปรโตคอลและเฟิร์มแวร์ ไม่ใช่ลายเซ็นดิจิทัล (cryptographic attestation) จึงยืนยันได้ว่าชิปตอบสนองเหมือน BNO08x ที่มีเฟิร์มแวร์จากโรงงาน แต่ไม่ใช่ว่าการพิสูจน์แหล่งที่มาแบบเด็ดขาด

11.2 การใช้งาน Factory Test

1. ต่อเพียง **3Vo**, **GND**, **SDA** → GPIO 21, **SCL** → GPIO 22 (ESP32) ไม่ต้องต่อ **INT** / **RST** (ESP32-S3 ใช้ SDA 14 / SCL 15)
2. วางบอร์ดนิ่งบนโต๊ะ ห่างจากแม่เหล็ก
3. อัปโหลด **10_Factory_Test** หรือเขียนเฟิร์มแวร์สำเร็จรูป (หัวข้อ 11.3)
4. เปิด Serial Monitor 115200 การทดสอบเริ่มอัตโนมัติหลังบูต พิมพ์ **r** แล้ว Enter เพื่อทดสอบซ้ำ

ผลจริงบน ESP32 DevKit + BNO086

```
#MASSMORE_FACTORY_TEST v1.0
#PRODUCT Massmore_BNO08x
#MCU ESP32
Library v2.1.1
#RESULT BUS_SCAN PASS 0x4A
#RESULT CHIP_ID PASS 10004563
#CHIP BNO086
#RESULT FW_VERSION PASS 3.12.6
#RESULT SERIAL PASS NONE
#RESULT METADATA PASS Q14/12
#RESULT AUTHENTICITY PASS GENUINE
#RESULT RANGE_ACCEL PASS 9.61
#RESULT RANGE_GYRO PASS 0.000
#RESULT RANGE_MAG PASS 48.9
#RESULT RANGE_QUAT PASS 1.000
#RESULT CONTINUOUS PASS 20/20
#VERDICT PASS
[PASS] SENSOR QA PASSED - READY TO SHIP
```

ตารางที่ 11.2 ความหมายและเกณฑ์ของแต่ละรายการ

รายการ	สิ่งที่ตรวจสอบ	เกณฑ์ผ่าน	เหตุผลเมื่อไม่ผ่าน
BUS_SCAN	สแกน I2C หา 0x4A / 0x4B	พบ	NO_DEVICE_ON_I2C, BOOTLOADER_MODE_BT_ PIN_LOW
CHIP_ID	part number ใน Product ID	10003606 / 10004095 / 10004563	NO_PRODUCT_ID_RESPON SE, CHIP_ID_MISMATCH
#CHIP	รุ่นชิป	ข้อมูลเท่านั้น ไม่มีผลต่อ ผลรวม	—
FW_VERSION	version ของเฟิร์มแวร์	major 1–9 และ build ≠ 0	FW_VERSION_IMPLAUSI BLE
SERIAL	FRS 0x4B4B	อ่านได้ หรือ NONE	SERIAL_READ_TIMEOUT
METADATA	Q point ของ Rotation Vector	14 / 12	MOTIONENGINE_METADA TA
AUTHENTICITY	รวมผลด้านบน	GENUINE	AUTHENTICITY_SUSPEC T
RANGE_ACCEL	a ขณะวางนิ่ง	$9.81 \pm 1.5 \text{ m/s}^2$	ACCEL_OUT_OF_RANGE
RANGE_GYRO	\omega ขณะวางนิ่ง	$\leq 0.5 \text{ rad/s}$	GYRO_OUT_OF_RANGE
RANGE_MAG	B	5–200 μT	MAG_OUT_OF_RANGE
RANGE_QUAT	q	1 ± 0.02	QUATERNION_NOT_UNIT
CONTINUOUS	อ่านต่อเนื่อง 20 ครั้ง	20/20 และ noise ของ $ a \leq 1.0$	NO_SENSOR_REPORTS

ทุกบรรทัดที่เว็บอ่านขึ้นต้นด้วย # รูปแบบ #RESULT <รายการ> <PASS|FAIL> <ค่า> และสรุปด้วย #VERDICT <PASS|FAIL> [เหตุผล] เพียงครั้งเดียว ตัวอ่านผลต้องข้าม บรรทัด # ที่ไม่รู้จักได้ (เช่น #CHIP)

11.3 เฟิร์มแวร์สำเร็จรูป

ตารางที่ 11.3 ไฟล์ในโฟลเดอร์ firmware/

ไฟล์	ตำแหน่ง เขียน	การใช้งาน
bin/ Massmore_BNO08x_FactoryTest_ESP32 .bin	0x0	ไฟล์รวม (bootloader + partition + app) ใช้กับ esptool หรือ web flasher
bin/ Massmore_BNO08x_FactoryTest_ESP32 _app.bin	0x10000	เฉพาะ application
manifest.json	—	สำหรับ ESP Web Tools

เขียนเฟิร์มแวร์ด้วย esptool (Windows ใช้ --port COM5 เป็นต้น)

```
esptool.py --chip esp32 --port /dev/cu.usbserial-0001 --baud 921600 \
  write_flash -z 0x0 bin/Massmore_BNO08x_FactoryTest_ESP32.bin
```

ถ้าเขียนไม่ผ่าน (Invalid head of packet) ให้ลด --baud เป็น 460800 หรือ 115200 เฟิร์มแวร์ build ด้วย PlatformIO env esp32dev (Arduino-ESP32 Core 3.3.11) flash 4 MB โหมด DIO

บทที่ 12

สรุป API ของไลบรารี

12.1 การเริ่มต้น

ตารางที่ 12.1 ฟังก์ชันเริ่มต้นและสถานะ

ฟังก์ชัน	คำอธิบาย	คืนค่า
<code>begin(addr, TwoWire&, intPin=-1, rstPin=-1)</code>	เริ่มต้นบน I2C (sketch เรียก <code>wire.begin()</code> เอง) ไม่พบ address จะลองอีกค่าให้	<code>bool</code>
<code>beginSPI(cs, int, rst, wake, SPIClass&, hz)</code>	เริ่มต้นบน SPI Mode 3 — ต้องมีทุกขา	<code>bool</code>
<code>beginUART(Stream&, intPin=-1, rstPin=-1)</code>	SHTP over UART 3 Mbaud	<code>bool</code>
<code>getI2CAddress()</code>	address ที่พบจริง	<code>uint8_t</code>
<code>lastError() / statusToString(code)</code>	รหัสข้อผิดพลาดล่าสุด / ข้อความอธิบาย	<code>status_t / const char*</code>

12.2 Simple Blocking API

ตารางที่ 12.2 ฟังก์ชันแบบรอผล

ฟังก์ชัน	คำอธิบาย	คืนค่า
<code>readAll(reading_t&, timeoutMs=300)</code>	อ่านครบชุด (RV + accel + gyro + mag) เปิด report ให้เองที่ 50 Hz ถ้ายังไม่เปิด	<code>bool</code>
<code>readEulerDeg(euler_t&, timeoutMs)</code>	roll / pitch / yaw เป็นองศา	<code>bool</code>
<code>readHeadingDeg(timeoutMs)</code>	yaw 0..360°	<code>float</code> หรือ <code>NAN</code>
<code>waitForReport(sensorId, timeoutMs)</code>	รอ report ชนิดที่ระบุ	<code>bool</code>

12.3 Non-blocking FSM API

ตารางที่ 12.3 ฟังก์ชันแบบไม่รอผล

ฟังก์ชัน	คำอธิบาย
<code>enableRotationVector(us), enableGameRotationVector(us), enableAccelerometer(us) ...</code>	เปิด report (argument เป็นคาบเวลา μs เช่น 10000 = 100 Hz)
<code>update() / updateAll(max=16)</code>	ดึง packet หนึ่งชุด / ทุกชุดที่ค้าง (ไม่รอ)
<code>isDataReady(id) / hasNewReport(id)</code>	ตรวจ report ใหม่ / ตรวจแล้วล้าง flag

ฟังก์ชัน	คำอธิบาย
<code>getReadings(reading_t&)</code>	คัดลอกค่าล่าสุดทั้งหมด
<code>getQuaternion(), getEulerDeg(), getHeadingDeg(), getAccel(), getGyro(), getGyroDeg(), getMag(), getLinearAccel(), getGravity()</code>	อ่านค่าล่าสุดแต่ละชนิด
<code>getAccuracy(id) / getQuatAccuracy()</code>	accuracy 0–3 ของ sensor / ค่าคลาดเคลื่อน heading (เรเดียน)
<code>setReportCallback(cb, ctx)</code>	เรียกฟังก์ชันที่กำหนดทุกครั้งที่มี report

12.4 Calibration, Tare, พลังงาน และ FRS

ตารางที่ 12.4 ฟังก์ชันอื่น ๆ

ฟังก์ชัน	คำอธิบาย
<code>calibrateAll(), calibrateMagnetometer(), endCalibration()</code>	เปิด / ปิด dynamic calibration
<code>saveCalibration(), clearCalibrationAndReset()</code>	บันทึก DCD ลง flash / ลบแล้วรีเซ็ต
<code>requestCalibrationStatus(), getCalibrationStatus(), calibrationComplete()</code>	ถามและอ่านสถานะ calibration
<code>tareNow(axes, basis), persistTare(), clearTare()</code>	ตั้งศูนย์ / บันทึก / ยกเลิก tare
<code>softReset(), hardwareReset(), modeSleep(), modeOn(), wake()</code>	รีเซ็ตและจัดการพลังงาน
<code>readFrsRecord(), writeFrsRecord(), readSensorMetadata()</code>	Flash Record System
<code>Massmore_BNO08x_RVC: begin(Stream&), read(rvc_report_t&), getChecksumErrors()</code>	คลาสสำหรับโหมด UART-RVC

12.5 รหัสข้อผิดพลาด

ตารางที่ 12.5 รหัสสถานะและข้อความจาก `statusToString()`

รหัส	ข้อความ	สาเหตุที่พบบ่อย
<code>MASSMORE_BNO08X_OK (0)</code>	—	สำเร็จ
<code>ERR_IO (-1)</code>	Bus I/O error (I2C held low? use RST pin or power-cycle)	bus ค้าง สายหลวม
<code>ERR_TIMEOUT (-2)</code>	Timeout	ชิปไม่ตอบทันเวลา
<code>ERR_BAD_PARAM (-3)</code>	Bad parameter	เช่น ไม่ส่ง <code>wakePin</code> ให้ <code>beginSPI()</code>
<code>ERR_NO_DEVICE (-4)</code>	No device found	ต่อสายผิด โหมดไม่ตรง

รหัส	ข้อความ	สาเหตุที่พบบ่อย
ERR_BAD_RESPONSE (-5)	Bad response	packet ผิดรูปแบบ
ERR_NOT_READY (-6)	Not initialised - call begin() first	ยังไม่เรียก <code>begin()</code>
ERR_UNSUPPORTED (-7)	Unsupported on this transport	คำสั่งใช้กับโหมดนี้ไม่ได้
ERR_WRONG_ID (-8)	Product ID does not match a BNO08x	ไม่ใช่ชิป BNO08x

บทที่ 13 การแก้ไขปัญหา

13.1 ปัญหาการเชื่อมต่อ

ตารางที่ 13.1 อาการ สาเหตุ และวิธีแก้ไข

อาการ	สาเหตุ	วิธีแก้ไข
No device found	ไฟ / GND / SDA-SCL สลับ, ขาในโค้ดไม่ตรง, BT ถูกดึง LOW	ตรวจสอบสายตามหัวข้อ 4.2 รันตัวอย่างสแกน I2C ด้านล่าง
Bus I/O error หรือ SDA / SCL อ่านได้ LOW ค้าง	MCU ถูกรีเซ็ตขณะชิปกำลังส่งข้อมูล ชิปกด bus ไว้	ต่อ RST แล้วส่ง rstPin ให้ begin() หรือถอดไฟเซนเซอร์ ≈ 5 วินาที
พบอุปกรณ์ที่ 0x28 / 0x29	ชิปอยู่ใน bootloader	ปล่อย BT ลอยแล้วรีเซ็ต
เปลี่ยน P0 / P1 แล้วโหมดไม่เปลี่ยน	โหมดถูกอ่านตอนรีเซ็ตเท่านั้น	ต่อ RST หรือถอดไฟเซนเซอร์
SPI: No device found ทั้งที่ต่อครบ	P0 ต่อ 3Vo หรือบัดกรี jumper PS0	ต่อ P0 เข้า GPIO แล้วส่งเป็น wakePin
UART-SHTP ไม่ตอบ	TX / RX สลับ, baud ผิด, buffer เล็ก	P1=1 P0=0, SDA → RX, SCL ← TX, 3,000,000 baud, setRxBufferSize(1024)
UART-RVC ไม่มีข้อมูล	ต่อ SDA ผิดขา หรือชิปยังไม่เข้าโหมด RVC	P1=0 P0=1 แล้วรีเซ็ตชิป, SDA → RX ของ MCU

โปรแกรมสแกน I2C (ESP32)

```
#include <Wire.h>
void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  Wire.setClock(100000);
  for (uint8_t a = 1; a < 127; ++a) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) Serial.printf("Found 0x%02X\n", a);
  }
}
void loop() {}
```

13.2 ปัญหาค่าที่อ่านได้

ตารางที่ 13.2 ปัญหาด้านข้อมูล

อาการ	สาเหตุ	วิธีแก้ไข
yaw กระโดดเมื่อมอเตอร์ทำงาน	สนามแม่เหล็กรบกวน (ไม่ใช่ปัญหา I2C)	ใช้ Game RV, ย้ายเซนเซอร์ห่างมอเตอร์ / สายกระแสสูง, calibrate ใหม่หลังประกอบ

อาการ	สาเหตุ	วิธีแก้ไข
yaw drift ช้า ๆ	ใช้ Game RV ซึ่งไม่มี magnetometer	ปกติ — กด z ตั้งศูนย์ใหม่ หรือใช้ RV + calibrate
มุมสลับไปมาระหว่างสองค่า	เปิด RV และ Game RV พร้อมกัน	เปิด report ทิศทางเพียงชนิดเดียว
tare แล้วมุมไม่เปลี่ยน	basis ของ <code>tareNow()</code> ไม่ตรงกับ report ที่เปิด	Game RV ใช้ <code>TARE_BASIS_GAMING_RV</code>
ข้อมูลกระตุกหรือหยุด	report มาก อัตราสูง หรือ <code>delay()</code> ยาว	ลดจำนวน / อัตรา report, ใช้ <code>updateAll()</code> , ต่อ INT, ใช้ SPI
heading ไม่ชี้ทิศเหนือ	ใช้ <code>headingDeg</code> จาก <code>readAll()</code> หรือยังไม่ calibrate	ใช้สูตรในบทที่ 8 และ calibrate ตามบทที่ 7
มุมกลับทิศหรือแกนสลับ	การติดตั้งบอร์ดไม่ตรงแกนอ้างอิง	ตรวจสอบจุดศูนย์กลาง X / Y / Z บนบอร์ด หมุน +90° ที่ละแกนเพื่อยืนยัน

13.3 ปัญหาจากเครื่องมือพัฒนา

ตารางที่ 13.3 ปัญหาที่ไม่เกี่ยวกับไลบรารี

อาการ	สาเหตุ	วิธีแก้ไข
Upload ESP32: <code>Invalid head of packet</code>	USB-UART หนึ่งที่ 921600	ลด Upload Speed เป็น 512000 หรือ 460800
<code>Resource busy</code> / อัปโหลดไม่ได้	Serial Monitor เปิดค้าง	ปิด Serial Monitor ก่อนอัปโหลด
MOMO: Serial Monitor ไม่แสดงข้อความ	USB CDC On Boot = Enabled	ตั้งเป็น Disabled
Nano อัปโหลดไม่ขึ้น	bootloader เก่า (57600)	เลือก ATmega328P (Old Bootloader) หรือ <code>board = nanoatmega328</code>
PlatformIO หา header ไม่พบ	เปิดโฟลเดอร์ผิดระดับ	เปิดโฟลเดอร์ <code>PlatformIO/</code> ที่มี <code>platformio.ini</code> แล้ว Rebuild IntelliSense Index

13.4 รายการตรวจสอบก่อนนำไปใช้ในผลิตภัณฑ์

- ☐ ยืนยันแรงดันไฟ ระดับลอจิก และ I2C address หลังประกอบ
- ☐ ต่อ INT / RST และทดสอบที่ 100 kHz ก่อนเพิ่มเป็น 400 kHz
- ☐ เลือกชนิด Rotation Vector ให้เหมาะกับงาน และ calibrate หลังติดตั้งในกล่องจริง (Save DCD)
- ☐ ทดสอบขณะมอเตอร์ทำงาน และทดสอบการรีเซ็ต ไฟตก ถอดเสียบไฟซ้ำ
- ☐ จำกัดอัตราการพิมพ์ log ไม่ให้รบกวนการอ่านข้อมูล
- ☐ บันทึกกรุ่นชิป เวอร์ชันเฟิร์มแวร์ address ขาที่ใช้ และขั้นตอน calibration ไว้ในเอกสารการผลิต

ภาคผนวก ก

ขาที่ตัวอย่างแต่ละชุดใช้

ตารางที่ ก.1 ขาที่กำหนดในไฟล์ .ino ของเวอร์ชัน 2.1.1

ตัวอย่าง	ESP32 DevKit	ESP32-S3 (MOMO)	Arduino Nano
01_BasicRead	SDA 21, SCL 22, RST 17	SDA 14, SCL 15 (RST 18*)	A4, A5
02_CustomPins_BusRemap	Wire1 SDA 21, SCL 22, INT 4, RST 17	Wire1 SDA 14, SCL 15 (RST 18*)	A4, A5, INT D2
03_GameRotationVector_Euler	SDA 21, SCL 22 (RST 17*)	SDA 14, SCL 15 (RST 18*)	A4, A5
04_NonBlocking_Multitask	SDA 21, SCL 22, INT 4, RST 17	SDA 14, SCL 15 (RST 18*)	A4, A5, INT D2
05_Calibration_Tare	SDA 21, SCL 22, RST 17	SDA 14, SCL 15 (RST 18*)	A4, A5
06_I2C_Euler_Compass	SDA 21, SCL 22, RST 17	SDA 14, SCL 15 (RST 18*)	A4, A5
07_SPI_Advance	SCK 18, MISO 19, MOSI 23, CS 5, INT 4, RST 17, WAKE 16	SCK 12, MISO 13, MOSI 11, CS 10, INT 4, RST 5, WAKE 6	D13, D12, D11, CS D10, INT D2, RST D3, WAKE D4
08_UART_RVC	RX 21 (TX 22 ไม่ใช้), RST 17	RX 14 (TX 15 ไม่ใช้)	SoftwareSerial RX D2
09_UART_Mode	RX 21, TX 22, INT 4, RST 17	RX 14, TX 15 (RST 18*)	ไม่รองรับ
10_Factory_Test	SDA 21, SCL 22	SDA 14, SCL 15 (RST 18*)	A4, A5

* ขาที่อยู่ในบรรทัดคอมเมนต์ เปิดใช้เมื่อต่อสาย RST แล้ว ทุกตัวอย่างเลือกขาตามชนิด MCU อัตโนมัติด้วย `#if defined(CONFIG_IDF_TARGET_ESP32S3)` และ `ARDUINO_ARCH_ESP32`

ภาคผนวก ข ผลการทดสอบและประวัติการเปลี่ยนแปลง

ข.1 MCU ที่รองรับ

ตารางที่ ข.1 ความเข้ากันได้ของ MCU

MCU	Core / Toolchain	การย้ายขา	ข้อจำกัด
ESP32-S3	Arduino-ESP32 3.x (IDE 3.3.12 / pioarduino 3.3.11)	GPIO Matrix ทุกขา	ไม่มี เหมะกับงาน SPI 400 Hz ขึ้นไป
ESP32 (Classic)	Arduino-ESP32 3.x	GPIO Matrix ทุกขา	ไม่มี เป็น MCU หลักของ Factory Test
Arduino Nano (ATmega328P)	Arduino AVR Core	ขาดายตัว (I2C A4/A5, SPI D10–13)	SRAM 2 KB ไลบรารีใช้ ≈ 1 KB ใช้ Simple API, ขาอื่นนอกจาก SDA/SCL ต้องแปลงระดับแรงดัน, ไม่รองรับ UART-SHTP

ข.2 ผลทดสอบกับบอร์ดจริง

ตารางที่ ข.2 ผลทดสอบ (BNO086 FW 3.12.6 part 10004563)

MCU	I2C	SPI	UART-SHTP	UART-RVC
ESP32 DevKit	PASS (01–06, 10)	PASS ≈ 400 Hz	PASS 469/469	PASS
ESP32-S3 MOMO	PASS (01, 04, 05, 06, 10)	ยังไม่ทดสอบ	ยังไม่ทดสอบ	ยังไม่ทดสอบ
Arduino Nano	compile เท่านั้น	compile เท่านั้น	ไม่รองรับ	compile เท่านั้น

ESP32 DevKit ทดสอบวันที่ 22 กันยายน 2569 (Arduino-ESP32 2.0.17) และตัวอย่าง 03 ทดสอบในวันที่ 6 ตุลาคม 2569 ด้วย PlatformIO (Core 3.3.11) ทำงานต่อเนื่อง 5 นาทีไม่ค้าง · ESP32-S3 MOMO ทดสอบวันที่ 27 กันยายน 2569 ด้วย PlatformIO (Core 3.3.11) · การ compile ตัวอย่าง 10 ชุด $\times$ 3 บอร์ด ผ่านครบ 30/30 ไม่มี warning (-Wall -Wextra)

ข.3 ประวัติการเปลี่ยนแปลง

ตารางที่ ข.3 สรุปการเปลี่ยนแปลงตามเวอร์ชัน

เวอร์ชัน	วันที่	การเปลี่ยนแปลงหลัก
2.1.1	27 ก.ย. 2569	ตัวอย่างใหม่ <code>03_GameRotationVector_Euler</code> (วัดมุมไม่ต้อง calibrate) และเรียงลำดับตัวอย่างใหม่เป็น 10 ชุด ตัวอย่าง Calibration ไม่เปิด RV กับ Game RV พร้อมกัน, บรรทัดสถานะบอกขั้นตอนถัดไป, ระบุ tare basis ชัดเจน ทุกตัวอย่างเพิ่มคอมเมนต์ <code>RST_PIN 18</code> สำหรับ ESP32-S3 · <code>clearTare()</code> เป็นวิธียกเลิก tare (<code>softReset()</code> ไม่ล้าง)
2.1.0	22 ก.ย. 2569	ใหม่: <code>getChipModel()</code> / <code>chipModelToString()</code> และตัวอย่าง UART-SHTP, Euler Compass แก้ I2C: มุมเพี้ยนเป็นครั้งคราวเมื่อ packet ยาว, probe ช้าเมื่อชิป NACK, แจ้ง <code>Bus I/O error</code> ทันทีเมื่อ bus ค้าง แก้: เวอร์ชันเฟิร์มแวร์ที่รายงานคงที่, UART ใช้งานร่วมกับ INT ได้, <code>readAll()</code> บน UART ไม่ timeout หลัง <code>delay()</code> SPI: <code>beginSPI()</code> บังคับ Wake
2.0.0	—	ปรับ API ใหม่ทั้งหมด: Simple Blocking API + Non-blocking FSM API, ตรวจสอบตัวต้นชิป, Factory Test แบบ <code>#RESULT</code> / <code>#VERDICT</code>

ภาคผนวก ค เอกสารอ้างอิงและแหล่งข้อมูล

ค.1 ไบรารีและสินค้า

- GitHub: github.com/Massmore/Massmore_BNO08x_SKU-1010
- หน้าสินค้า: massmore.shop — Halley BNO085 / BNO086
- บทความคู่มือสินค้า: massmore.shop/docs/28
- Shopee: shopee.co.th/i.5641091.49802229903 · Lazada: lazada.co.th/products/i4774958834.html

ค.2 เอกสารจากผู้ผลิตชิป

- [BNO08X Datasheet \(1000-3927\)](#)
- [SH-2 Reference Manual \(1000-3625\)](#)
- [BNO08X Sensor Calibration Procedure \(1000-4044\)](#)
- [BNO08X Tare Function Usage Guide \(1000-4045\)](#)
- [BNO080 / BNO085 Product Brief](#)

ค.3 เครื่องมือพัฒนา

- [Arduino IDE](#) · การติดตั้งไลบรารีใน Arduino IDE
- [Arduino-ESP32 installation guide](#)
- [PlatformIO IDE for VS Code](#) · [pioarduino platform-espressif32](#)

ค.4 บทความวิชาการพื้นฐาน

- Kok, Hol และ Schön, [Using Inertial Sensors for Position and Orientation Estimation](#) — drift, แบบจำลองเซนเซอร์ และวิธี fusion
- Sabatini, [Kalman-Filter-Based Orientation Determination Using Inertial/Magnetic Sensors](#) — quaternion EKF และผลของสนามแม่เหล็กบกพร่อง
- Madgwick, [An efficient orientation filter for inertial and inertial/magnetic sensor arrays](#)